NN Builder

User manual: using the application, designing a neural network,
and the demo architectures explained

The NN Builder project
https://github.com/varunjha089/NN-Builder

November 2026 (version 0.2 and the changes after it)

Abstract

NN Builder is a desktop application for designing neural networks on a canvas, the way one draws a block diagram. The design is checked while it is drawn: every connection shows the shape of the tensor it carries, every block shows its output size and parameter count, and mistakes are explained in plain words on the block itself. A finished design becomes a complete, readable PyTorch training script that also loads the data (a built-in dataset, Hugging Face or Kaggle), and the application runs it on a GPU somewhere else: a free Kaggle notebook, a college GPU server over SSH, or an Amazon EC2 instance (including spot instances, which may be interrupted). The computer the application runs on only designs: it never trains, never downloads datasets and needs no PyTorch. This manual explains the application screen by screen, walks through designing a network from scratch, gives the arithmetic behind every number the application shows, explains the fifteen demo architectures (from LeNet-5 to a one-billion-parameter Vision Transformer, plus one original design), and ends with troubleshooting. References point to the original papers, on arXiv wherever they are published there.

Contents

1 What NN Builder is **4**

1.1 The idea 4

1.2 The workflow at a glance 4

1.3 What it is not 4

2 Installing, updating and the window **4**

2.1 Installing 4

2.2 Automatic updates 5

2.3 The top bar 5

2.4 Menus and keyboard shortcuts 5

2.5 Day and night themes, Settings 6

3 Projects **6**

3.1 Creating, opening, saving 6

3.2 Demo architectures are unsaved until you save them 8

3.3 What a project folder contains 8

4 The Design screen **8**

4.1 Layout . . . 8
4.2 Working with blocks . . . 8
4.3 The layer table . . . 9
4.4 Famous architectures and the skip-connection tool . . . 10

5 Designing a network, step by step **10**
5.1 When something is wrong . . . 11

6 The arithmetic behind the numbers **11**
6.1 Output size . . . 12
6.2 Computation . . . 12
6.3 Parameters . . . 12
6.4 Receptive field . . . 12
6.5 Memory needed for training . . . 13

7 Reference of all blocks **13**
7.1 Transformers in NN Builder . . . 14
7.2 Segmentation . . . 14

8 The Dataset tab **15**
8.1 Built-in datasets . . . 15
8.2 Hugging Face . . . 15
8.3 Kaggle . . . 16
8.4 Segmentation data . . . 16
8.5 Image size and classes . . . 16

9 The Code tab: what the generated script does **16**

10 The Connection tab **17**
10.1 Machines . . . 18

11 Training **18**
11.1 Where to train . . . 18
11.2 Training settings . . . 18
11.3 Following a run . . . 19
11.4 Several seeds, comparing runs, exporting results . . . 20
11.5 Independence from the connection . . . 20
11.6 EC2 spot instances . . . 21
11.7 Can my GPU train this network? . . . 21

12 The Results tab **21**

13 The demo architectures **22**
13.1 LeNet-5 (1998) . . . 22

13.2 Small CNN for CIFAR-10 . . . 23
13.3 AlexNet (2012) . . . 23
13.4 VGG-16 (2014) . . . 24
13.5 GoogLeNet / Inception v1 (2014) . . . 24
13.6 ResNet-18 (2015) . . . 24
13.7 MobileNet v1 (2017) . . . 25
13.8 DenseNet, small CIFAR version (2017) . . . 25
13.9 Transformer encoder on image rows (2017) . . . 25
13.10 ViT-Base/16 (2020) . . . 26
13.11 ViT-Tiny for CIFAR-10 . . . 26
13.12 ViT-g/14, one billion parameters (2022) . . . 26
13.13 U-Net (2015) . . . 27
13.14 Attention U-Net (2018) . . . 27
13.15 MSG-Former: an NN Builder original . . . 28
13.15.1 The idea . . . 28
13.15.2 The multi-scale gated residual block . . . 28
13.15.3 The whole network . . . 29
13.15.4 Training it . . . 32
13.15.5 Experiments to run (ablations) . . . 32
13.15.6 Adapting it . . . 32
13.15.7 Status . . . 33

14 Troubleshooting **33**

15 Privacy and security **33**

A Glossary **34**

1 What NN Builder is

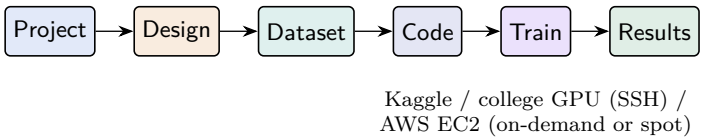
1.1 The idea

Building a neural network usually means writing code, running it, and finding out from an error message (or worse, from silently bad results) that a layer received a tensor of the wrong shape. NN Builder moves that discovery to the moment of design. It is inspired by visual programming tools such as MIT App Inventor: blocks are placed and connected, and the tool refuses or explains combinations that cannot work.

Three principles shape the application:

1. **The design computer only designs.** Shapes, parameter counts and the generated code are computed by an engine written in Rust inside the application, instantly and offline. No deep-learning library is installed locally, and no dataset is downloaded to the design computer.
2. **Training happens on a GPU elsewhere, automatically.** One button generates the script, uploads it, prepares the machine if needed, starts the run, follows it, and brings the results back.
3. **A run does not depend on the connection.** On a server or an EC2 instance the run is started detached; it continues when the laptop sleeps, the network drops or the application is closed, and the application reconnects to it later.

1.2 The workflow at a glance



The tabs in the top bar follow this order. **Connection**, set apart on the right, holds the tokens and machines that training needs, and is usually visited once.

1.3 What it is not

NN Builder covers image classification and image segmentation. It does not (yet) train on text, audio or tabular data, and it does not run training on the design computer, even for a tiny network. The generated script, however, is ordinary PyTorch [29]: it can be copied, read, edited and run anywhere by hand.

2 Installing, updating and the window

2.1 Installing

Installers are published with every release on GitHub (`https://github.com/varunjha089/NN-Builder/releases/latest`):

- **macOS** (Apple silicon): the file ending in `_aarch64.app.zip`. Unzip it and move `NN Builder.app` to the Applications folder. The application is not signed by Apple, so the first start may need a right-click on the application and **Open**.
- **Windows**: `NN-Builder_X_x64-setup.exe`.

- **Linux**: `nn-builder_X_amd64.deb` or the portable `.AppImage`.

2.2 Automatic updates

Once a day (if enabled), the application asks GitHub for the latest release. When a newer version exists, a banner appears under the top bar with **Install and restart**. The download is checked against the SHA-256 checksum published with the release before anything is replaced. The switch is in **Settings** (and on the Projects page); **NN Builder** → **Check for Updates...** checks at once.

2.3 The top bar

From left to right: the name, the **Project** button (the current project, or *No project*; it opens the Projects page), a **not saved** badge and a **Save...** button when the open project is an unsaved demo, the tabs **Design**, **Dataset**, **Code**, **Train**, **Results**, then on the right a summary ("*N params · ready*" in green, or "*design has problems*" / "*dataset incomplete*" in red), the day/night button (a moon in day mode, a sun in night mode), the **Settings** button, and **Connection** (a red dot means something is still missing there).

2.4 Menus and keyboard shortcuts

Menu	Item (shortcut)	What it does
NN Builder	Check for Updates...	asks GitHub for a newer release
	Settings... (Cmd+,)	opens the Settings page
File	New Project... (Cmd+N)	Projects page, cursor in the name field
	Open Project... (Cmd+O)	choose a project folder
	Save Project... (Cmd+S)	keeps an unsaved demo as a real project
	New from Demo ▷	opens one of the fifteen demo architectures
	Import / Export Design...	reads or writes a design (`model.json`)
	Export train.py... (Cmd+E)	saves the generated training script
	Show Project in Finder (Shift+Cmd+R)	reveals the project folder
Edit	Undo (Cmd+Z), Redo (Shift+Cmd+Z)	undo/redo changes to the design (in a text field: the typing)
	Arrange Left to Right (Cmd+L)	lays the canvas out in columns by depth
View	Night Mode (Shift+Cmd+N)	switches the neon night theme
Connection	Open Connection (Shift+Cmd+K)	the Connection page
	Install Tools...	installs the `kaggle` and `boto3` helpers

2.5 Day and night themes, Settings

The night theme draws panels dark and boxes and text as neon glows; every layer glows in the colour of its kind (Conv2D orange, pooling teal, normalisation violet, Dense blue, joins red, outputs green, transformer blocks teal and violet). In night mode the Design screen is split into three separate panels (blocks on the left, canvas in the middle, inspector on the right). Their background colours are chosen in **Settings** → **Night panel colours**: six presets (Midnight, Deep purple, Ocean, Carbon, Forest, Crimson) or a colour picker per panel, with a live preview; a warning appears for colours so light that neon text becomes hard to read. The theme, the panel colours and the automatic-update switch are all on the Settings page (Figure 1); Figure 2 shows the day theme.

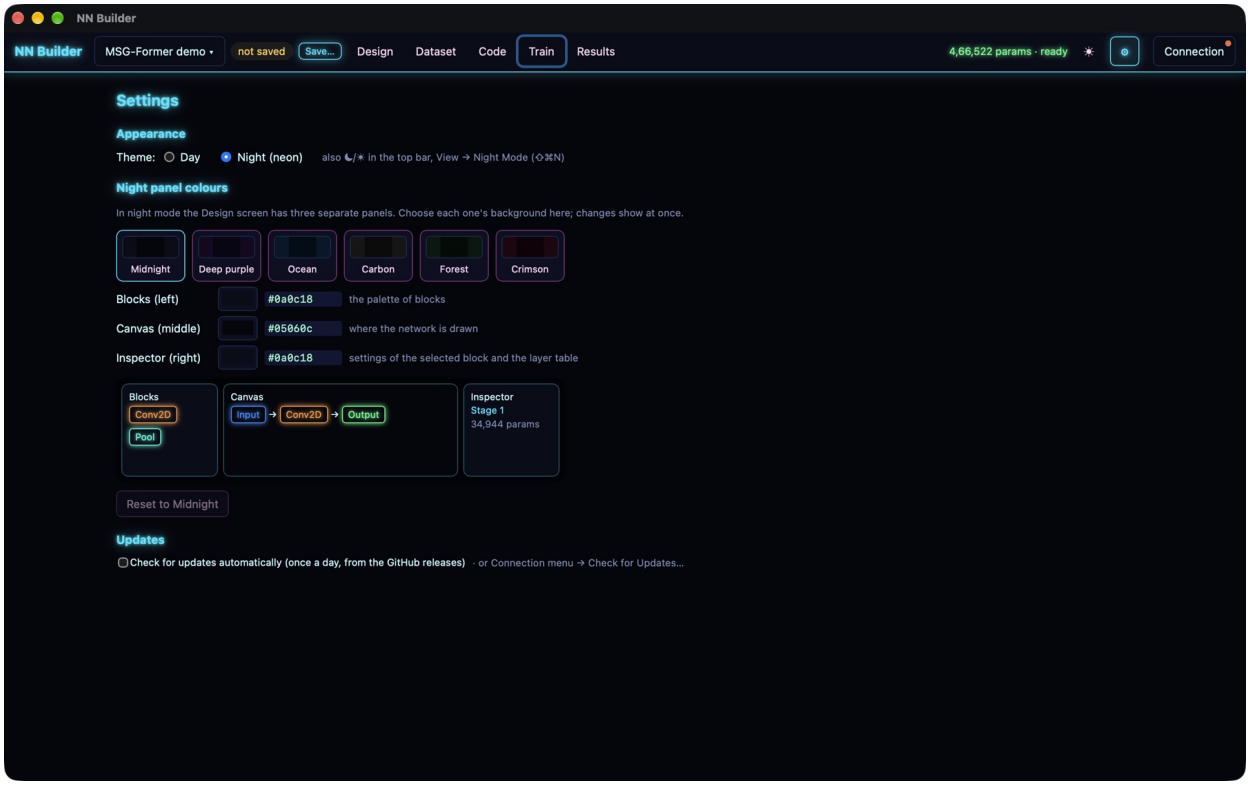


Figure 1: Settings: the theme, the six presets and a colour picker per panel for the night Design screen, a live preview, and the automatic-update switch.

3 Projects

3.1 Creating, opening, saving

A *project* is a folder. The Projects page has two cards side by side:

- **New project**: type a name and press **Create** (or Enter). The folder is made in the projects folder shown underneath (**change** picks another one).
- **Open**: the projects used before, each with its folder, and a field to open any project folder by its path.

Every change is saved as it is made; there is no "save" step for a normal project.

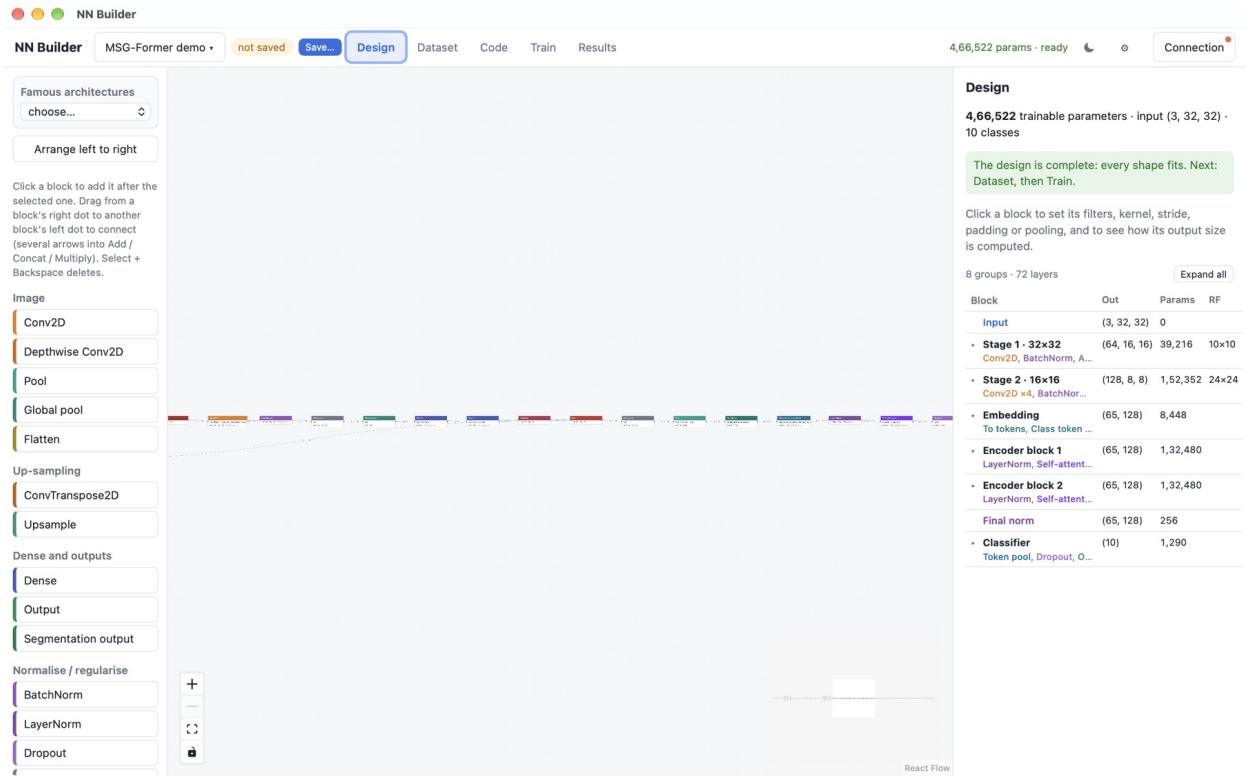


Figure 2: The Design screen in the day theme (MSG-Former, fitted to the canvas; the layer table on the right).

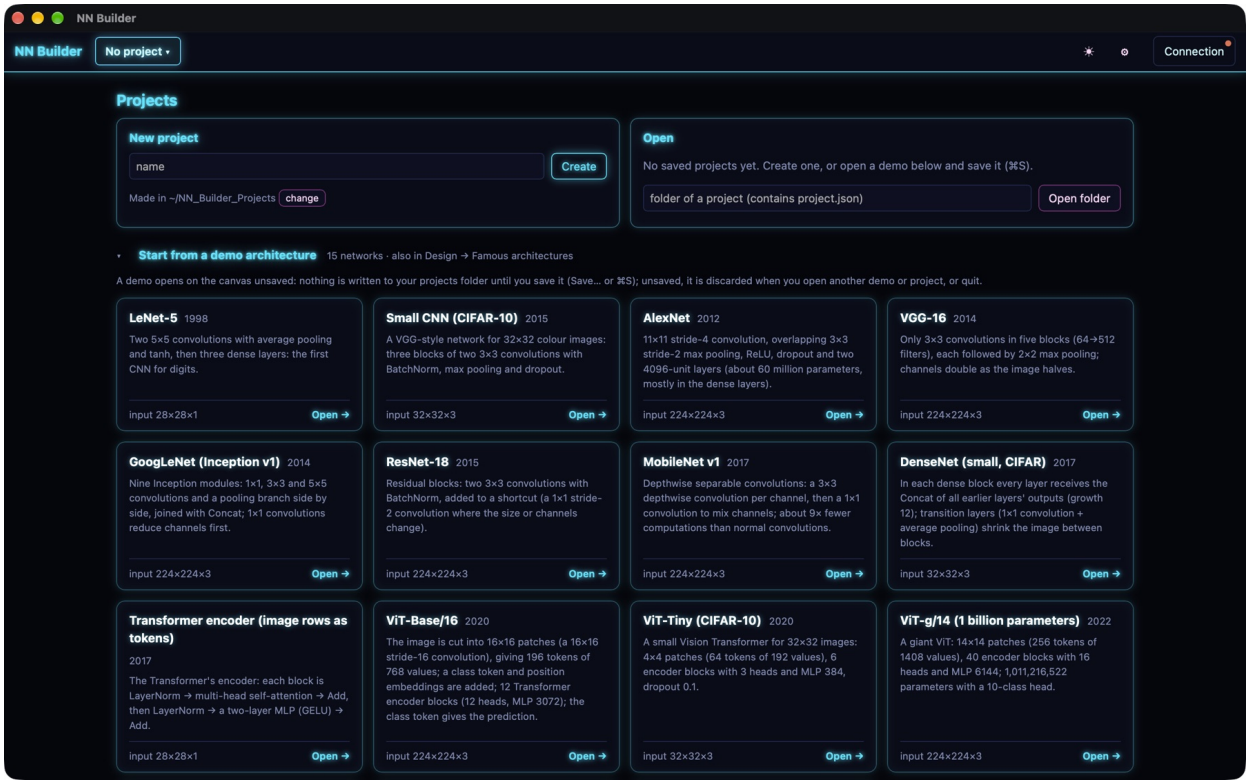


Figure 3: The Projects page (night theme): New project and Open side by side, then the demo architectures as tiles.

3.2 Demo architectures are unsaved until you save them

Below the two cards, **Start from a demo architecture** shows the fifteen demos as tiles (the section folds with its arrow). Clicking a tile opens that network on the canvas as an *unsaved* project: nothing is written to the projects folder. The top bar then shows **not saved** and **Save...**. Saving (**Save...** or Cmd+S) asks for a name and moves it into the projects folder as a normal project. An unsaved demo is discarded, without asking, when another demo or project is opened or created, and when the application quits. This keeps the projects folder free of "AlexNet demo 2, 3, 4...". A demo cannot be trained before it is saved, so that runs always belong to a kept project.

3.3 What a project folder contains

`project.json`	name, creation date
`model.json`	the design: blocks (with their settings and positions) and arrows
`dataset.json`	the dataset choice
`train.json`	the training settings
`runs/<id>/`	one folder per training run (code, metrics, log, results, model)

A design can also be exported on its own (**File** → **Export Design...**) and imported into another project.

4 The Design screen

4.1 Layout

Blocks (left). The palette, grouped as *Image* (Conv2D, Depthwise Conv2D, Pool, Global pool, Flatten), *Up-sampling* (ConvTranspose2D, Upsample), *Dense and outputs* (Dense, Output, Segmentation output), *Normalise / regularise* (BatchNorm, LayerNorm, Dropout, Activation), *Transformer* (To tokens, Class token + position, Self-attention, Token pool) and *Branches* (Add, Concat, Multiply). Above them: the **Famous architectures** picker and **Arrange left to right**.

Canvas (middle). The network, read from left to right. Each block shows its title (with × for an error and a warning sign for a warning), a one-line summary of its settings, and its output shape with its parameter count. Each arrow shows the shape of the tensor it carries. Zoom and pan with the trackpad or the controls; a minimap sits in the corner.

Inspector (right). With a block selected: its settings, its shape formula with its own numbers, its receptive field, the sliding-window animation (convolution and pooling), and the skip-connection tool. With nothing selected: the whole design's parameter count, its problems, and the layer table.

4.2 Working with blocks

- **Add:** click a block in the palette; it is added after the selected block (and connected to it).
- **Connect:** drag from a block's right dot to another block's left dot. Add, Concat and Multiply accept several arrows; all other blocks take one.
- **Delete:** select a block or an arrow and press Backspace. The Input block cannot be deleted.
- **Move:** drag blocks; **Arrange left to right** (Cmd+L) places them in columns by depth (the longest path from the Input), stacking parallel branches in their column.

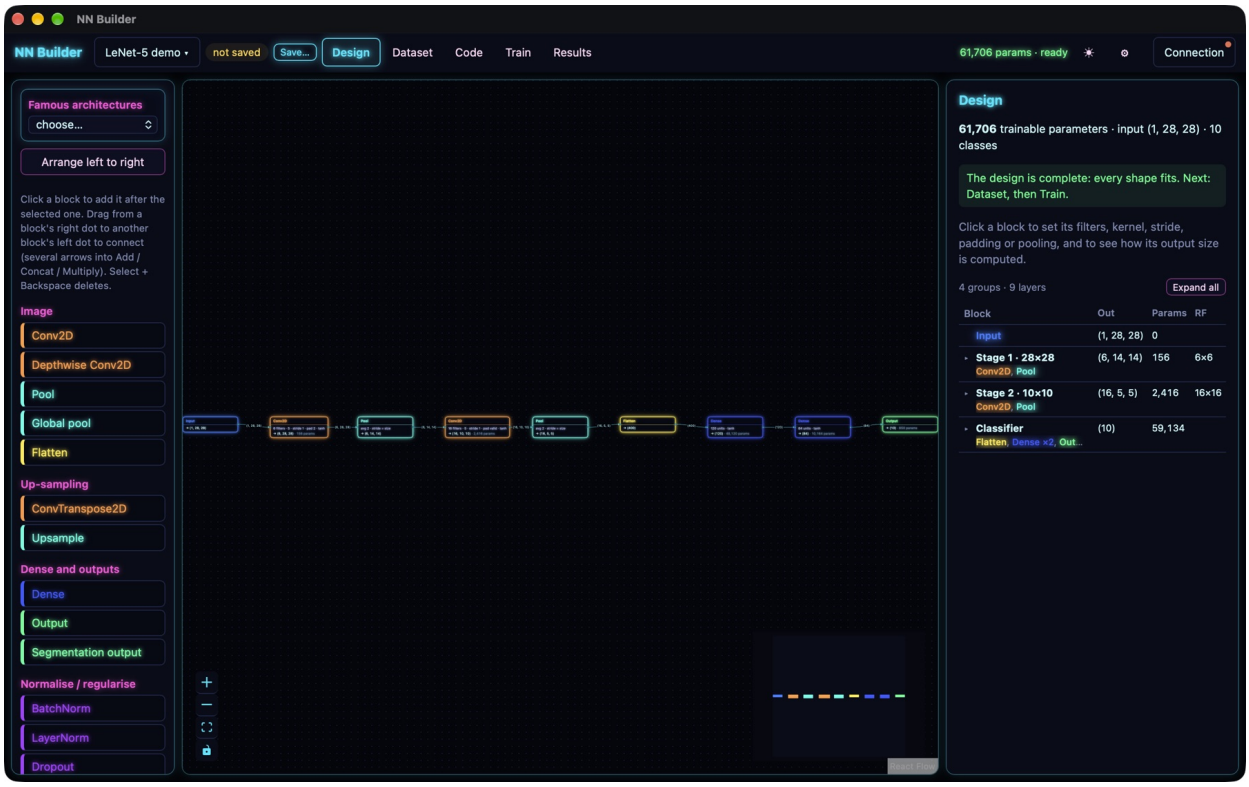


Figure 4: The Design screen with LeNet-5: blocks on the left, the network on the canvas (every arrow labelled with its tensor shape, every block with its output and parameters), and on the right the whole design's summary and the grouped layer table.

- **Undo / Redo:** `Cmd+Z` / `Shift+Cmd+Z`; changes made within 0.6 seconds of each other count as one step.
- **Copy / paste:** select blocks (Shift-click or drag a box with Shift) and use `Cmd+C` / `Cmd+V`: the copies, with the arrows between them, appear below the originals, ready to be connected.
- **Residual block after selected:** inserts Conv 3×3 → BatchNorm → ReLU → Conv 3×3 → BatchNorm → Add (with a skip from the selected block) → ReLU, keeping its size and channels.
- **Collapse repeated blocks (×N):** draws identical consecutive groups as one node, for example the 40 encoder blocks of ViT-g/14 as "Encoder block ×40" (287 blocks become 8). It is a read-only view; double-click the node or press **Show every block** to edit again.

4.3 The layer table

With nothing selected, the inspector lists the layers grouped into stages, each with its output shape, its parameter subtotal and the receptive field at its end:

- **Input**.
- **Image stages** by resolution ("Stage 2 · 27×27"); a pooling layer, a Dropout after it, and small vector layers inside a stage (a squeeze-and-excitation gate) stay with their stage; decoder stages that grow the image are marked with an up arrow.
- **Transformers**: *Embedding* (with a patch convolution before it), *Encoder block 1, 2, …*, *Final norm*.
- **Classifier**: the vector layers at the end.

Each group shows its contents in block colours ("Conv2D ×3, Pool") and opens with a click (**Expand all** opens all).

4.4 Famous architectures and the skip-connection tool

Famous architectures replaces the current design with a demo (the previous design is kept as `model.previous.json`) and adjusts the dataset's image size to the one the architecture was made for. The skip-connection tool (inspector, with a block selected) joins an earlier block's output to the selected block's output with *Add* (a residual connection, as in ResNet [9]) or *Concat* (as in DenseNet [15] and U-Net [32]). When the shapes differ, it inserts a projection: a 1×1 convolution with the right stride (images), a pooling layer, or a Dense layer (vectors).

5 Designing a network, step by step

This section builds a small convolutional network for CIFAR-10 [18] (32×32 colour photos, 10 classes).

1. A project and the data first. Create a project "CIFAR CNN". Open **Dataset**, choose **Built-in (torchvision)** and **CIFAR-10**. The Input block now says $(3, 32, 32)$: three colour channels of 32×32 pixels. (Choosing the data first means every shape shown while designing is the real one.)

2. A convolution. In **Design**, select the Input block and click **Conv2D**. In the inspector set **Filters** to 32 and choose the preset "3×3, stride 1 (keep size)". The block shows $\rightarrow (32, 32, 32)$ and 896 parameters ($3 \cdot 3 \cdot 3 \cdot 32$ weights + 32 biases). The arrow from Input shows $(3, 32, 32)$.

3. Downsampling. With the convolution selected, click **Pool**: max pooling 2×2, stride "same as pool size". Output $(32, 16, 16)$, no parameters. The inspector animates the 2×2 window sliding with stride 2: every window position gives one output value.

4. Repeat, wider. Add another Conv2D (64 filters, 3×3, same) and Pool: $(64, 8, 8)$. A third pair with 128 filters gives $(128, 4, 4)$. This is the common pattern: *the image gets smaller and deeper.*

5. From image to vector. A classifier needs one vector per image. Two choices:

- **Flatten**: $(128, 4, 4) \rightarrow (2048)$. Keeps the spatial layout, but the next Dense layer then has many weights.
- **Global pool** (average): $(128, 4, 4) \rightarrow (128)$, one number per channel [23]. Far fewer parameters, and it works for any image size.

Choose Global pool here.

6. The head. Add **Dense** (128 units, ReLU), **Dropout** (rate 0.3) [12, 36], then **Output**. The Output block reads the number of classes from the dataset (10) and has $128 \cdot 10 + 10 = 1290$ parameters. The top bar now says "*... params · ready*".

7. Make it better.

- Insert **BatchNorm** [16] after each convolution (set the convolution's activation to *none*, and add an **Activation** ReLU after the BatchNorm): faster, more stable training.
- Select the second convolution and use the skip-connection tool to add its input to its output (*Add*): a residual block. Where the channel counts differ, the tool adds the 1×1 projection for you.
- Look at the layer table: it now shows Stage 1 · 32×32, Stage 2 · 16×16, Stage 3 · 8×8, Classifier, each with its parameter subtotal.

8. Check, then train. The **Code** tab shows the generated script; **Train** starts it on Kaggle, a server or EC2 (Section 11).

5.1 When something is wrong

A block turns **red** when the design cannot work (training is blocked) and **yellow** when it works but probably not as intended. Blocks after a red block are greyed out ("fix the layer above"); blocks not connected to the Input are faded and ignored. The reasons are written on the block and in the inspector. The checks include:

Level	Check (as the application words it, shortened)
error	exactly one Input; exactly one Output (or Segmentation output), and it must be last; no loops
error	the image is too small for the window ("use a smaller kernel, padding, or fewer pooling layers before this")
error	Dense / Output on an image: "add Flatten or Global pool before it"
error	an image block (convolution, pooling) after Flatten or after To tokens
error	Add with different shapes ("use a 1×1 Conv2D to match channels, or a stride to match sizes")
error	Concat with different heights/widths; Multiply with incompatible shapes
error	attention: the token dimension must divide evenly by the number of heads
error	BatchNorm on tokens (use LayerNorm); Output on tokens (add Token pool)
error	segmentation: the output must return to the mask size ("Upsample / ConvTranspose2D")
error	pooling padding more than half the window; output padding not smaller than the stride (PyTorch rules)
error	the dataset has no classes yet
warning	the stride does not divide the input evenly: the last rows/columns are ignored
warning	stride larger than the window: pixels between windows are skipped
warning	"same" padding with an even kernel cannot keep the size exactly
warning	a very large Dense layer after Flatten ("try Global pool instead")
warning	a block whose result is not used; Flatten on an input that is already flat

6 The arithmetic behind the numbers

The engine computes everything below exactly as PyTorch does; on every change to the code, an automatic test on GitHub builds hundreds of random designs (and all demos) in real PyTorch and compares every shape and parameter count with the application's (most recently: 296 designs, 0 differences). A good visual guide to this arithmetic is Dumoulin and Visin [7].

6.1 Output size

For a convolution or pooling layer with input size n (height or width), kernel k, stride s, padding p and dilation d:

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d\,(k-1) - 1}{s} \right\rfloor + 1. \tag{1}$$

- **same**: padding chosen so that, with stride 1, $n_{\text{out}} = n$ (for an odd kernel, $p = d(k-1)/2$). With stride $s > 1$ the application computes the explicit padding PyTorch needs (it refuses `padding="same"` with a stride), giving $n_{\text{out}} = \lceil n/s \rceil$.
- **valid**: $p = 0$.
- **a number**: that many pixels of zeros on each side.

Dilation [44] spreads the kernel: a 3×3 kernel with dilation 2 covers 5×5 pixels with 9 weights. For a transposed convolution (up-sampling):

$$n_{\text{out}} = (n-1)\,s - 2p + d\,(k-1) + p_{\text{out}} + 1, \tag{2}$$

so a 2×2 kernel with stride 2 exactly doubles the size. Upsample multiplies the size by its scale.

6.2 Computation

The Design screen's summary also gives the **multiply–accumulate operations (MACs) per image**: convolutions (weights × output positions), dense layers (per token on a sequence) and attention ($4Nd^2 + 2N^2d$); normalisation, activations, pooling and joins are counted as zero. ResNet-18 at 224 × 224 comes to the usual 1.8 G.

6.3 Parameters

Block	Parameters	Example
Conv2D	$k_h k_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}$ (bias)	3×3, 64→128: 73 856
Depthwise Conv2D	$k_h k_w C_{\text{in}} m + C_{\text{in}} m$	3×3, 512 channels: 5 120
ConvTranspose2D	$k_h k_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}$	2×2, 512→256: 524 544
Dense / Output	$n_{\text{in}} n_{\text{out}} + n_{\text{out}}$	4096→4096: 16 781 312
BatchNorm	$2C$ (scale and shift)	64 channels: 128
LayerNorm	$2D$	768 values: 1 536
Self-attention	$4d^2 + 4d$ (query, key, value, output)	$d = 768$: 2 362 368
Class token + position	$d + (N+1)\,d$	196 tokens, $d = 768$: 152 064
Pool, Global pool, Flatten, Dropout, Activation, joins	0	

Convolutions followed by BatchNorm are created without a bias in the demos (the BatchNorm shift makes it redundant), as torchvision does.

6.4 Receptive field

The receptive field of a block is the size of the input region that one of its output values can see. With $r_0 = 1$ and $j_0 = 1$ (the "jump", the distance in input pixels between neighbouring outputs), each layer with kernel k, dilation d and stride s gives

$$r_\ell = r_{\ell-1} + d\,(k-1)\,j_{\ell-1}, \qquad j_\ell = j_{\ell-1}\,s. \tag{3}$$

The inspector shows it for every block ("one output value here sees 99×99 input pixels"). A network should usually reach a receptive field about the size of the objects it must recognise; the *effective* receptive field is smaller and concentrated in the centre [25].

6.5 Memory needed for training

A useful estimate for training with Adam [17] in mixed precision [27] is about **16 bytes per parameter** before any images: 4 for the weights, 4 for the gradients and 8 for Adam's two running averages. On top come the activations kept for the backward pass, which grow with the batch size and the image size. Examples: ResNet-18 (11.2 M parameters) needs about 0.2 GB for its state; ViT-Base (86 M) about 1.4 GB; ViT-g/14 (1.01 B) about 16 GB, more than a 16 GB T4 has. If a run fails with "CUDA out of memory", halve the batch size; beyond that, gradient checkpointing [3] and sharding the optimizer state over several GPUs [30] are the standard remedies (not built into NN Builder).

7 Reference of all blocks

Block	Shape in → out	Settings and notes
Input	→ (C, H, W)	set by the dataset (channels, height, width)
Conv2D	(C, H, W) → (F, H', W')	filters, kernel (with a "square" link: unlink for height×width), stride, padding (same / valid / number), dilation, activation, bias; presets: 3×3 keep size, 3×3 stride 2 (halve), 1×1 (mix channels), 5×5, 7×7 stride 2 (ResNet stem)
Depthwise Conv2D	(C, H, W) → (Cm, H', W')	one filter per input channel (channel multiplier m); with a following 1×1 Conv2D it forms a depthwise separable convolution [13]
ConvTranspose2D	(C, H, W) → (F, H'', W'')	learnt up-sampling; filters, kernel, stride, padding, output padding
Pool	(C, H, W) → (C, H', W')	max or average; pool size; stride (default: same as size); padding
Global pool	$(C, H, W) \to (C)$	average or max over the whole image
Upsample	(C, H, W) → (C, sH, sW)	scale, nearest or bilinear; no parameters
BatchNorm	same shape	normalises each channel over the batch [16]; images and vectors only
LayerNorm	same shape	normalises each sample [1]; on tokens, over the dimension; on an image, over (channels, height, width)
GroupNorm	same shape	normalises each image over groups of channels; independent of the batch, so good for small batches; the channels must divide by the number of groups (often 32)
Dropout	same shape	rate: the fraction of values set to zero during training [36]
Flatten	$(C, H, W) \to (CHW)$	
Dense	$(n) \to (u)$; tokens $(N, D) \to (N, u)$	units, activation; on tokens it acts on each token (the transformer MLP)
Activation	same shape	ReLU, Leaky ReLU, GELU [11], SiLU [31], sigmoid, tanh

Block	Shape in → out	Settings and notes
Add	equal shapes → same	element-wise sum: residual connections [9]
Concat	joins along channels	images need equal height and width; the arrow order is the channel order
Multiply	see notes	equal shapes; a one-channel map times an image (attention gate [28]); a per-channel vector times an image (squeeze-and-excitation [14])
To tokens	$(C, H, W) \to (N, D)$	*patches*: every pixel of the feature map becomes a token of C values ($N = HW$), used after a patch convolution as in ViT [6]; *rows*: every image row is a token
Class token + position	$(N, D) \to (N{+}1, D)$	a learnt class token in front, and learnt position embeddings added
Self-attention	$(N, D) \to (N, D)$	heads, dropout; multi-head scaled dot-product attention [40]
Token pool	$(N, D) \to (D)$	the class token, or the mean of all tokens
Output	$(n) \to (K)$	K = number of classes; softmax/cross-entropy is in the training script
Segmentation output	$(C, H, W) \to (K, H, W)$	a 1×1 convolution giving a class score for every pixel; must match the mask size

7.1 Transformers in NN Builder

A transformer works on a *sequence of tokens*, shape (N, D): N tokens of D values. To use one on images:

1. cut the image into patches with a convolution whose kernel equals its stride (ViT: 16×16, stride 16, D filters), giving $(D, H/16, W/16)$;
2. **To tokens** (patches) turns that map into $N = (H/16)(W/16)$ tokens;
3. **Class token + position** adds a learnt class token and position information (attention by itself ignores order);
4. repeat the *encoder block*: LayerNorm → Self-attention → Add (with the block's input), then LayerNorm → Dense (GELU, usually $4D$ units) → Dense (D) → Add. Normalising before each part ("pre-norm") trains more stably than after it [43];
5. a final LayerNorm, **Token pool** (class token), **Output**.

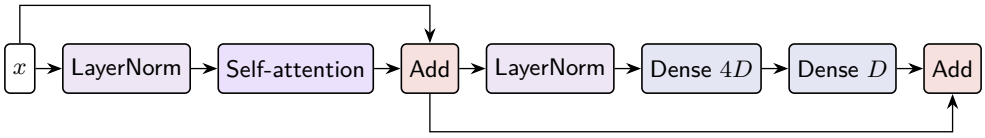


Figure 5: A pre-norm Transformer encoder block as it is built on the canvas.

7.2 Segmentation

For segmentation (Dataset → **Segmentation**) every pixel gets a class. The network must end with **Segmentation output** at the size of the masks, so an encoder that shrinks the image needs a decoder that grows it again (ConvTranspose2D or Upsample), as in U-Net [32]. The training script uses a per-pixel cross-entropy loss and reports pixel accuracy and the mean intersection over union (mIoU) of the classes.

8 The Dataset tab

Only the *description* of a dataset is read on the design computer; the images are downloaded on the GPU machine when training starts (and cached there in `~/.nn_builder/data`).

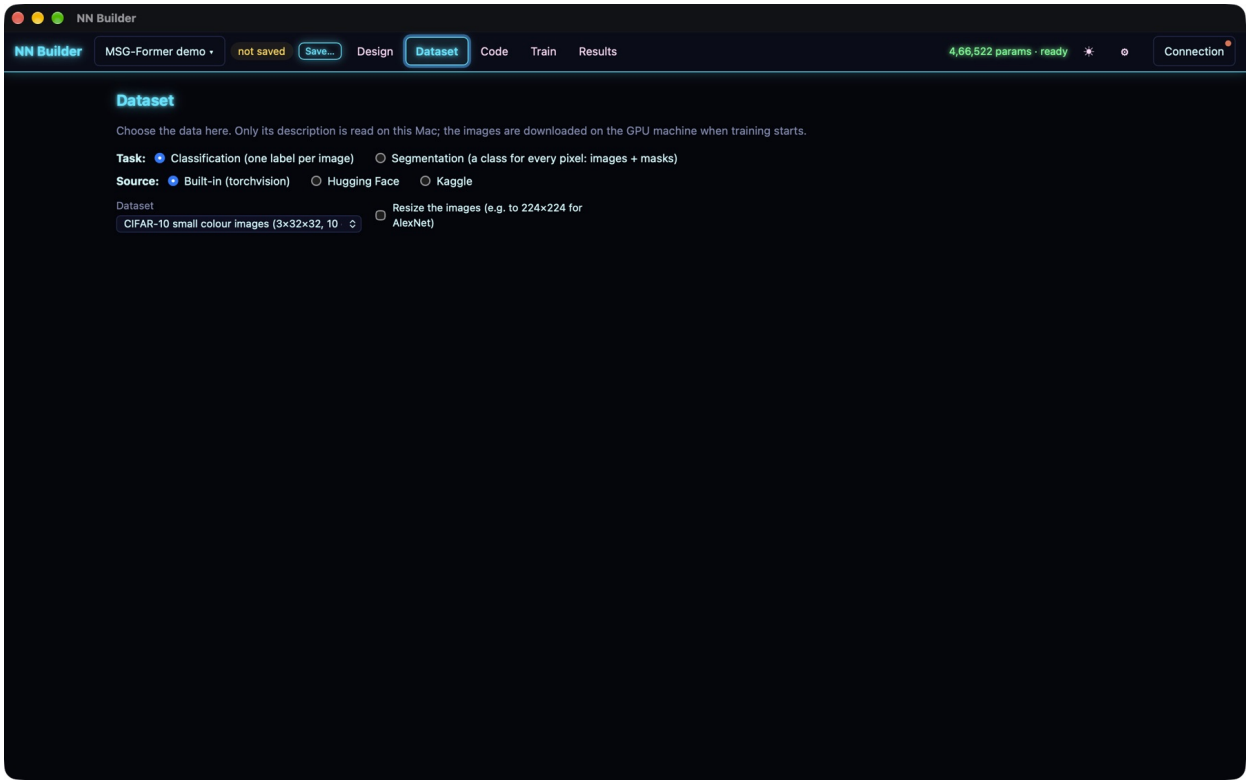


Figure 6: The Dataset tab with built-in CIFAR-10, as MSG-Former opens (the demo picks the built-in set that fits its input, so nothing is resized).

8.1 Built-in datasets

MNIST (handwritten digits, $1 \times 28 \times 28$) [21], Fashion-MNIST (clothing, $1 \times 28 \times 28$) [42] and CIFAR-10 (photos, $3 \times 32 \times 32$) [18], ten classes each, downloaded by torchvision on the GPU machine. **Resize the images** enlarges them for networks made for larger inputs (for example 224×224 for AlexNet); the number of channels can be changed as well (grey images repeated into three channels).

8.2 Hugging Face

Type a dataset id (`owner/name`) and press **Read info**: the application reads its configurations, splits, columns and first rows from the public datasets-server API, without downloading [22]. Then choose:

- **Config** and **Training split** (with its number of rows);
- **Test split**: a separate split, or "10% of the training split" (a fixed, seeded split);
- **Image column** and **Label column**: the column that holds the pictures (an *Image* feature) and the one that holds the class (a *ClassLabel* feature, whose class names fill the class list).

Gated or private datasets need a Hugging Face token (Connection tab), which the run can copy to the machine.

Only image datasets work. A text corpus such as `HuggingFaceFW/fineweb-edu` (web pages and their metadata) has no image column and no class column: the image and label columns then point at unrelated text, the class list stays empty, and the design is marked as having problems. Use datasets such as `uoft-cs/cifar10`, `ylecun/mnist` or `food101`. The Dataset tab now says so at once when a dataset has no image column.

After **Read info**, the tab also shows the **class balance** of the training split (counted by Hugging Face's dataset viewer) with a warning when the largest class has more than three times the images of the smallest, and **Show 8 example images** fetches just eight pictures to check that the right columns are chosen (the dataset itself is still downloaded only on the GPU machine).

8.3 Kaggle

Type the dataset slug (`owner/name`); with the Kaggle token stored, the application lists its files and recognises the common layout of *one folder per class*. Set the training folder (and optionally a test folder; otherwise 10% of the training images are kept for testing) and the class names (the folder names, in order). On Kaggle the dataset is attached to the notebook; on a server or EC2 it is downloaded with the Kaggle tool, which needs the token on that machine (the Train tab offers to copy it there).

8.4 Segmentation data

Hugging Face: an image column and a **Mask column**. Kaggle: an **Images folder** and a **Masks folder** whose files have the same names. The class list maps mask values to classes (value 0 is the first class; with two classes any non-zero pixel is the second).

8.5 Image size and classes

Image height, **Image width** and **Channels** set the shape of the Input block; images are resized to it. The class names, one per line in label order, set the Output block's size.

9 The Code tab: what the generated script does

The **Code** tab shows the complete `train.py`; **Copy** and **Export train.py** save it. It needs only PyTorch and torchvision (plus `datasets` or `kaggle` for those sources) and runs anywhere with `python train.py`. In order, it:

1. defines the network as an `nn.Module` whose `forward` follows the graph, and lists every layer's expected shape;
2. loads the data with the chosen transforms and augmentation (training images only), normalised with the dataset's statistics;
3. runs a **one-batch check** on the GPU machine: one batch forward and backward, comparing every layer's real shape with the application's, and stopping at once with a clear message on a difference;
4. trains with Adam [17], AdamW or SGD with momentum 0.9 and the chosen recipe (learning-rate schedule with warm-up, weight decay, label smoothing, gradient clipping), with automatic mixed precision on CUDA [27];
5. after each epoch evaluates on the test data and writes a **checkpoint** (`checkpoint.pt`; optionally also every few minutes inside an epoch); a restarted run **resumes** from it, in the middle of an epoch with the same batch order;

6. writes one JSON line per event to `metrics.jsonl` (start, batch, epoch, checkpoint, resume, interrupted, done, stopped, error) and finally `model.pt` (the weights) and `results.json` (test accuracy, confusion matrix, sample predictions, feature maps, first-layer filters, and per-class IoU for segmentation).

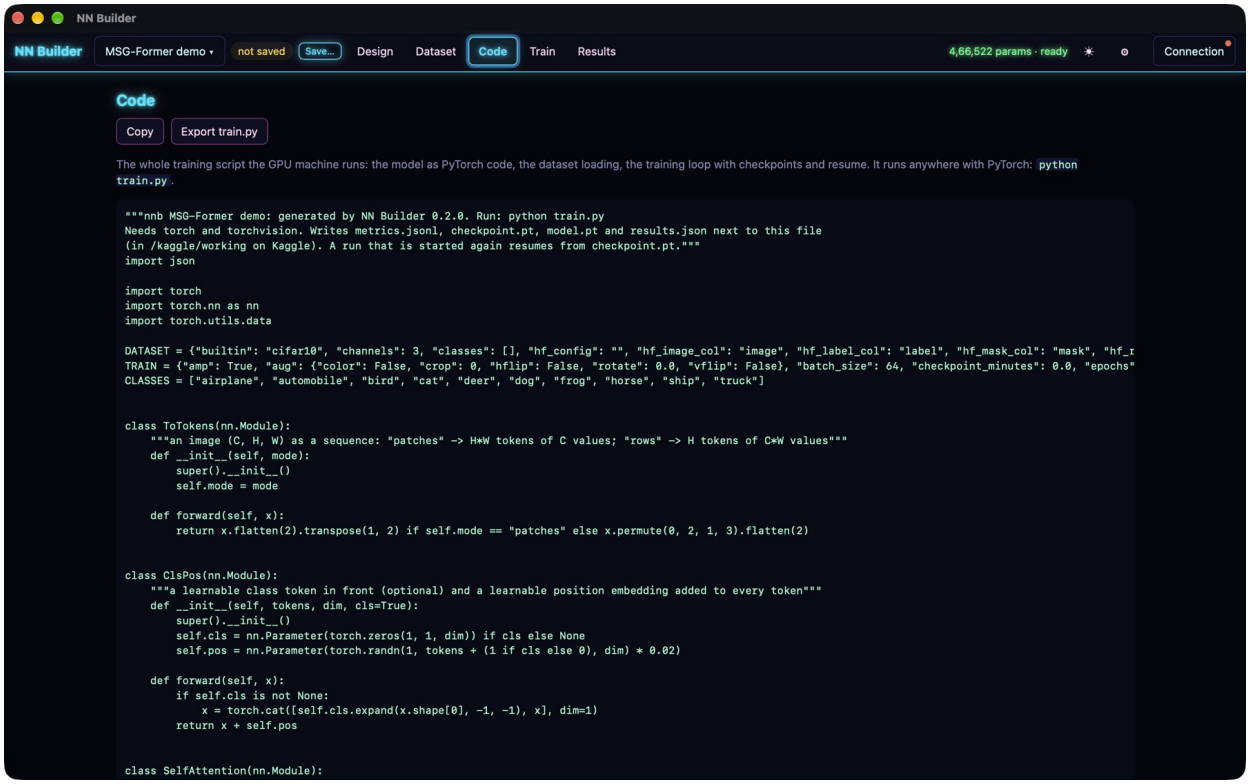


Figure 7: The Code tab: the generated `train.py` (here the start of MSG-Former's: the settings as Python literals and the transformer modules).

Signals: `TERM/INT` end the run cleanly (the model and results are still saved); on EC2, `USR1` means "AWS is taking the instance back" (checkpoint and exit to be resumed) and `USR2` means "checkpoint now" (Section 11.6).

10 The Connection tab

Tokens and keys are kept in the system keychain (macOS Keychain, Windows Credential Manager, Linux Secret Service) and handed to one call at a time; they are never written into projects, code or logs. When a run needs one on a remote machine, it is written there as a file readable only by its owner, through the SSH connection's input, never on a command line.

Kaggle. kaggle.com → profile → Settings → API → Create New Token. Paste the username and key (or the whole `kaggle.json`). GPU notebooks need a phone-verified account.

Hugging Face. Only for gated or private datasets: a "Read" token from huggingface.co → Settings → Access Tokens.

AWS. An IAM user's access key that may only start, stop and describe your instance (the page shows the policy to paste, with `ec2:DescribeSpotInstanceRequests` added for spot details).

Tools. **Install tools** makes a small private Python environment with the `kaggle` and `boto3` packages (about 60 MB) used to talk to Kaggle and AWS. No deep-learning library is installed.

10.1 Machines

College / any SSH server. Host (or an alias from `~/.ssh/config`), user, port, login by key or *password*, optional jump host, remote folder (default `.nn_builder`), optional existing Python with PyTorch. With a password, **Connect (log in once)** opens a Terminal window where you type it; the application then reuses that connection and never sees the password. **Test** checks the login; **Prepare** checks Python, the GPUs (`nvidia-smi`), the disk, and installs PyTorch in the remote folder (no administrator rights needed). **Keep runs going with** chooses how a run survives logging out: SLURM (`sbatch`, the right way on a shared cluster), `systemd-run` (needs "lingering"), `tmux`, or `nohup`+`setsid`; *automatic* picks the best the machine allows.

AWS EC2 instance. Region, instance id, user (`ubuntu`) and the instance's key file (`.pem`). **Check instance** reports its state, type and shutdown behaviour (it must be *Stop*, not *Terminate*, or a finished run would delete the instance), and for a spot instance its request type and interruption behaviour. The application starts the instance for a run and the run stops it at the end.

Keep a copy of checkpoints on this Mac. For either kind: every new checkpoint is copied back while the application is open, so a run can continue on another machine if its own machine is lost (essential for spot instances that may be terminated).

11 Training

11.1 Where to train

	Kaggle	**College GPU (SSH)**	**AWS EC2 g4dn.xlarge**
GPU	T4 or P100 (Kaggle's choice)	whatever the server has (e.g. RTX A6000 48 GB)	NVIDIA T4 16 GB, 4 vCPU
Cost	free (about 30 GPU hours a week)	free	per hour; spot is cheaper but may be interrupted
Limits	about 12 h per run; runs may queue	the server's rules (SLURM time limits)	your quota for G instances
Progress	from the notebook's log, every 30 s	live (every few seconds)	live
Needs	Kaggle token	SSH access	AWS key, key file, quota

11.2 Training settings

Setting	**Meaning and advice**
Train on	Kaggle, or a machine from the Connection tab
GPU on the machine	which GPU(s) to use on a shared server (empty = any free one)
Epochs	passes over the training data
Batch size	images per step; larger is faster but needs more memory (halve it on "out of memory")
Learning rate	0.001 suits Adam on small networks; very large networks (ViT-Base and bigger) need 0.0001 or less [37]
Optimizer	Adam [17], AdamW (weight decay applied separately [24]) or SGD with momentum 0.9 (often better for ResNet-style networks with a good schedule [8])

Setting	Meaning and advice
Training recipe	learning-rate schedule (constant; cosine decay to 0; or halve when the validation loss stalls for 3 epochs), linear warm-up over the first epochs, weight decay, label smoothing, gradient clipping. **Use recommended** sets AdamW, weight decay 0.05, 5 warm-up epochs then cosine, label smoothing 0.1 and clipping at 1.0, the usual recipe for transformers and a good one for CNNs [37, 8]. The defaults are plain constant-rate training, as in earlier versions
Training images (0 = all)	a quick trial on a subset
End the run after (hours of running)	a time limit, summed over resumes (protects the budget)
Checkpoint every (minutes)	also checkpoint inside an epoch; 0 = only at the end of each epoch (spot instances then use 10 minutes)
Automatic resumes	spot instances: how many interruptions are resumed automatically (default 20)
Validation part of the training data	held out from the training images (default 0%: the curves' "validation" is then the test data). Use 10–20% whenever results are reported: the test data is then touched only once, at the end
Early stopping	stop when the validation loss (or accuracy, or mIoU) has not improved for *patience* epochs; the best epoch's weights become `model.pt` and the test accuracy is measured on them. Switching it on sets a 10% validation part if there is none. Epochs is then the maximum
Random seed	fixes the initial weights, the derived splits and the batch order; run several seeds to report a mean and spread
Mixed precision	faster on T4/A100-class GPUs, half the activation memory [27]
Stop the EC2 instance	stop the instance when the run ends (saves money)
Data augmentation	horizontal / vertical flip, random crop after padding, rotation, brightness/contrast; training images only [4]. Flips change the meaning of digits; vertical flips suit aerial images
Copy my token to the machine	needed for Kaggle data on a server/EC2 and for gated Hugging Face data
Start from ImageNet weights	transfer learning: torchvision's pretrained ResNet-18, VGG-16 or ViT-B/16 loaded into the matching demo, tensor by tensor (a new output head for your classes starts at random); optionally train only the new head. Images are then normalised with ImageNet's statistics. Usually far better than training from scratch on small datasets; use a smaller learning rate (e.g. 0.0001) when fine-tuning everything

Above the Train button the tab says what is still missing for the chosen target (tools, token, key) and gives a rough estimate of the GPU memory the training needs (16 bytes per parameter plus the layer outputs at the chosen batch size), compared with the target's GPU (Kaggle and EC2 g4dn: 16 GB; an SSH machine: as **Prepare** found it). It warns when the design is likely not to fit.

11.3 Following a run

Train on ... starts a background task whose log appears at the bottom right. The run card then shows the state (starting, running with the epoch, interrupted, done, stopped, crashed, failed), the GPU, the training set size, the loss curves (per batch, training and validation per epoch) and the

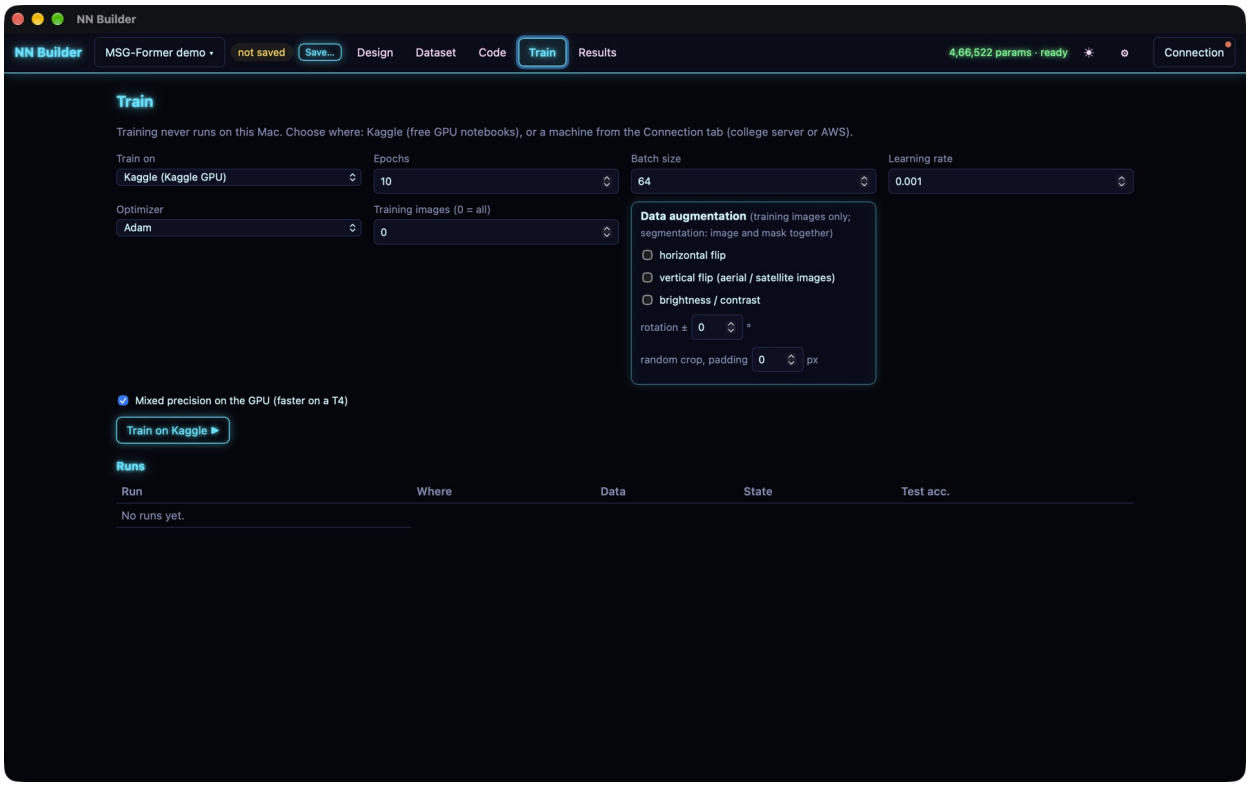


Figure 8: The Train tab: where to train, the main settings, augmentation, and the list of runs. Settings for SSH and EC2 machines (time limit, checkpoint interval, automatic resumes, stopping the instance) appear when such a machine is chosen.

accuracy curves (and mIoU for segmentation), the last checkpoint, and the log. **Stop** ends the run cleanly (the current epoch ends early, the model and results are saved). **Fetch results** copies the outputs back (for a stopped EC2 instance it starts the instance, copies, and stops it again). The **Runs** table lists every run of the project with its seed and test accuracy.

11.4 Several seeds, comparing runs, exporting results

A single training run is one sample: the same design and settings with another random seed give a slightly different accuracy. Next to the Train button, **seeds** starts 3 or 5 runs at once with seeds $s, s{+}1, \ldots$ (one group; on Kaggle one notebook per seed, which Kaggle may queue; on a server they share its GPU). Tick runs in the **Runs** table (or select a whole group of seeds) and press **Compare selected**: their validation curves are overlaid, and a table gives each run's recipe, epochs, best epoch, test accuracy (and top-5), training time, and for each group the mean $\pm$ sample standard deviation. **Export CSV + LaTeX** writes the table to the project's `exports` folder as `runs-<time>.csv` and `runs-<time>.tex` (a `booktabs` tabular with a "Mean $\pm$ s.d." row, ready to `\input` into a paper).

11.5 Independence from the connection

On a server or EC2, the run folder is copied with `rsync` and started detached (SLURM, systemd, tmux or nohup). The application only looks at it: every few seconds while the run is open it copies the new metric lines and the end of the log. Closing the application, sleeping the laptop or losing the network does not affect the run; the application catches up when it can reach the machine again.

11.6 EC2 spot instances

Spot instances use spare EC2 capacity at a lower price, but AWS may take one back with a two-minute notice. NN Builder recognises a spot instance by itself. The recommended setup is a *persistent* spot request with interruption behaviour *stop* (AWS restarts the instance when capacity returns, with its disk), plus **Keep a copy of checkpoints on this Mac**.

Situation	What NN Builder does
Two-minute interruption notice	a watcher on the instance sees it in the instance metadata; the script saves a mid-epoch checkpoint at once and exits to be resumed
Rebalance recommendation (earlier warning)	a checkpoint is saved now; training continues
Interruption without notice	at most the checkpoint interval of training is lost (10 minutes by default on spot)
Interrupted while writing a checkpoint, or disk full	checkpoints are written to a temporary file and swapped in: the previous one is never damaged
Instance restarted while the laptop is off	a boot-time job on the instance resumes the run: no laptop needed
Instance back while the application is open	the application follows the instance's new address and restarts the run if needed
Two starters at once	a lock lets only one copy of a run train
Resumed in the middle of an epoch	the remaining batches of that epoch, in the same order; optimiser, mixed-precision scaler and random state restored
Interrupted again and again	at most the set number of automatic resumes; the time limit counts running time over all resumes
Instance terminated	shown as "terminated"; with the copy on the Mac, **Continue on another machine** starts a new run from it (for example on a college GPU)
Old instance returns after moving	when that run is opened and the instance answers, the run there is stopped (cancel its spot request in the AWS console to be sure)
Stop while the instance is down	remembered: the run is stopped instead of resumed when the instance answers

11.7 Can my GPU train this network?

Use the 16-bytes-per-parameter rule of Section 6 plus room for activations. A 48 GB RTX A6000 can train ViT-g/14 (about 18 GB of fixed state) with batches of roughly 16 to 32 images at 224×224; a 16 GB T4 cannot. Large networks trained from scratch on small datasets overfit: ViT-g was trained on billions of images [45], and even ViT-Base needs strong augmentation and regularisation, or pre-training, to do well on small data [37, 39].

12 The Results tab

For a finished run (open it in the Train tab): the test accuracy (and top-5 accuracy with more than five classes, and mIoU), precision, recall and F1 of every class with their macro averages and the hardest classes, the confusion matrix (rows: true class, columns: predicted; the diagonal is correct), sample predictions with their confidence (wrong ones marked), the feature maps of the first test image (first eight channels of early layers), the first layer's filters as small pictures, and for segmentation the true and predicted masks and the IoU of every class; **Grad-CAM**

heat maps on the sample images (where the network looked: the last image-shaped layer's feature maps weighted by how strongly the predicted class depends on them [34]); and a gallery of up to 16 **misclassified** test images with their true and predicted class. **Show files** reveals `model.pt`, `train.py` and `results.json`, and with **Also save the trained model as TorchScript and ONNX** also `model.ts` (load with `torch.jit.load`, no NN Builder code needed) and `model.onnx` (for ONNX Runtime and other frameworks). The weights load in PyTorch with `Net().load_state_dict(torch.load("model.pt"))` using the `Net` class from the same `train.py`.

13 The demo architectures

Each demo opens unsaved on the canvas (Projects page, **File** → **New from Demo**, or **Famous architectures** in the Design tab) at the input size it was made for. Parameter counts below are the application's, with a 10-class output (2 classes for the segmentation networks); they change with the number of classes. Where a demo differs from the paper, the difference is stated.

Demo	Year	Task	Input	Main idea	Parameters
LeNet-5	1998	classify	1×28×28	convolution + subsampling	61 706
Small CNN (CIFAR-10)	2015	classify	3×32×32	VGG-style blocks + BatchNorm	551 018
AlexNet	2012	classify	3×224×224	deep CNN on GPUs, ReLU, dropout	58 322 314
VGG-16	2014	classify	3×224×224	only 3×3 convolutions	134 301 514
GoogLeNet	2014	classify	3×224×224	Inception modules	5 983 802
ResNet-18	2015	classify	3×224×224	residual connections	11 181 642
MobileNet v1	2017	classify	3×224×224	depthwise separable convolutions	3 217 226
DenseNet (small)	2017	classify	3×32×32	dense connectivity	76 810
Transformer encoder	2017	classify	1×28×28	self-attention on image rows	71 498
ViT-Base/16	2020	classify	3×224×224	image patches as tokens	85 806 346
ViT-Tiny (CIFAR-10)	2020	classify	3×32×32	a small ViT	1 806 538
ViT-g/14	2022	classify	3×224×224	scaling a ViT to 1 B parameters	1 011 216 522
U-Net	2015	segment	3×128×128	encoder–decoder with skips	7 760 130
Attention U-Net	2018	segment	3×128×128	attention gates on the skips	7 847 894
MSG-Former	2026	classify	3×32×32	NN Builder original (hybrid)	466 522

13.1 LeNet-5 (1998)

Background. LeCun et al. trained LeNet-5 to read handwritten digits on bank cheques [21]; it established the pattern every CNN since has followed: convolutions that detect local features, subsampling that makes them tolerant to small shifts, and fully connected layers that classify.

As built in NN Builder.

Block	Settings	Output	Parameters
Conv2D C1	6 filters, 5×5, padding 2, tanh	6×28×28	156
Pool S2	average 2×2	6×14×14	0
Conv2D C3	16 filters, 5×5, valid, tanh	16×10×10	2 416
Pool S4	average 2×2	16×5×5	0
Flatten		400	0
Dense C5, F6	120 then 84 units, tanh	84	48 120 + 10 164
Output	10 classes	10	850

Differences from the paper. The paper used 32×32 inputs with no padding (padding 2 on 28×28 MNIST gives the same 28×28 maps), trainable subsampling coefficients, a hand-designed partial connection table in C3, and Gaussian output units; the demo uses plain average pooling, full connections and a softmax output.

Training it. Dataset: built-in MNIST (no resizing needed). Adam, learning rate 0.001, batch 64, 5–10 epochs; it reaches about 99% test accuracy on any GPU in a few minutes. Look at the first-layer filters in Results: they become stroke and edge detectors.

13.2 Small CNN for CIFAR-10

Background. Not one paper but the standard "VGG-style" baseline for 32×32 colour images: stacks of 3×3 convolutions [35] with BatchNorm [16] and dropout [36].

As built. Three blocks with 32, 64 and 128 filters; each block is two times (Conv2D 3×3 without bias → BatchNorm → ReLU), then max pooling 2×2 and dropout (0.2, 0.3, 0.4). The image goes 32 → 16 → 8 → 4; Flatten gives 2 048 values, then Dense 128 (ReLU), Dropout 0.5 and the output. The classifier holds 263 562 of the 551 018 parameters.

Training it. Built-in CIFAR-10; Adam 0.001, batch 128, 30–50 epochs, with horizontal flips and a random crop with 4 pixels of padding: such networks commonly reach the mid to high 80s in test accuracy. Without augmentation the training accuracy runs far ahead of the validation accuracy (overfitting): compare the two curves in the Train tab.

13.3 AlexNet (2012)

Background. Krizhevsky, Sutskever and Hinton won the ImageNet challenge [33] of 2012 by a wide margin with a deep CNN trained on two GPUs [19]; its ingredients (ReLU instead of tanh, dropout, data augmentation, GPU training) started the deep-learning era in computer vision.

As built. Conv2D 96 filters 11×11 stride 4 padding 2 (224 → 55) → max pool 3×3 stride 2 (overlapping, 55 → 27) → Conv2D 256 5×5 → pool (13) → Conv2D 384, 384, 256 (3×3) → pool (6) → Flatten (9 216) → Dropout 0.5 → Dense 4 096 → Dropout 0.5 → Dense 4 096 → Output. Of the 58.3 million parameters, 37.8 million sit in the first Dense layer: the layer table's Classifier group shows this at a glance.

Differences from the paper. No local response normalisation, and no split of the layers into two groups (one per GPU); with 1 000 classes the demo has about 62 million parameters (the paper reports 60 million). A later note by Krizhevsky describes the single-GPU variant [20].

Training it. AlexNet expects 224×224 images: use CIFAR-10 with **Resize** (slow and not ideal: tiny photos enlarged) or a Hugging Face dataset of larger photos. Use a smaller learning rate (Adam 0.0001) and flips. It fits a T4 at batch 64–128.

13.4 VGG-16 (2014)

Background. Simonyan and Zisserman showed that depth matters and that small filters suffice: two 3×3 convolutions see as far as one 5×5, three as far as one 7×7, with fewer parameters and more non-linearities [35].

As built. Five blocks of 3×3 convolutions (ReLU, padding same) with 64, 64 | 128, 128 | 256 ×3 | 512 ×3 | 512 ×3 filters, each block followed by 2×2 max pooling (224 → 7); Flatten (25 088), two Dense 4 096 with dropout 0.5, Output. 134 301 514 parameters, exactly torchvision's `vgg16` with a 10-class head; 102.8 million of them are in the first Dense layer.

Training it. Like AlexNet: 224×224 images, Adam 0.0001 or SGD 0.01 with momentum, batch 32–64 on a T4. Watch the receptive field grow in the inspector: 3×3 convolutions and poolings reach 212×212 pixels at the last convolution.

13.5 GoogLeNet / Inception v1 (2014)

Background. Szegedy et al. asked which filter size to use, and answered "all of them, side by side": the Inception module runs 1×1, 3×3 and 5×5 convolutions and a pooling branch in parallel and concatenates them [38]. 1×1 convolutions first reduce the channels (the idea comes from Network in Network [23]), keeping the cost low: GoogLeNet has twelve times fewer parameters than AlexNet and was more accurate.

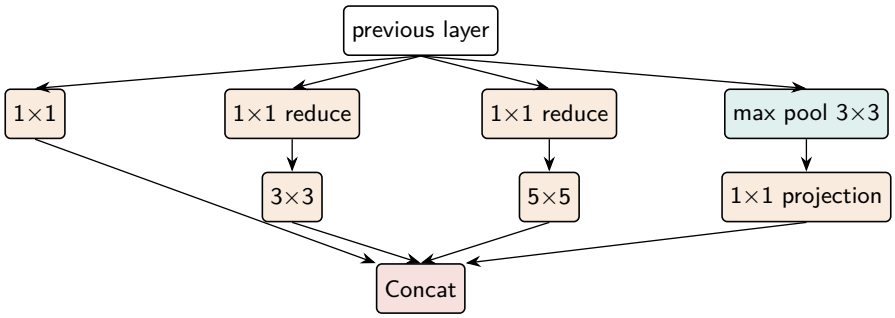


Figure 9: An Inception module: four branches side by side (all keep the height and width); Concat stacks their channels.

As built. Stem: Conv2D 64 7×7 stride 2, max pool, Conv2D 1×1 64, Conv2D 3×3 192, max pool (224 → 28). Then nine Inception modules with the paper's channel numbers: 3a (output 256 channels), 3b (480), max pool, 4a–4e (512, 512, 512, 528, 832), max pool, 5a (832), 5b (1 024); global average pooling, dropout 0.4, output. 5 983 802 parameters.

Differences from the paper. The two auxiliary classifiers used during training are left out (later work found them of little use), and there is no local response normalisation.

Training it. 224×224 inputs; without BatchNorm, deep plain networks like this train slowly: use Adam 0.0001–0.0003 and patience, or add BatchNorm after the convolutions yourself.

13.6 ResNet-18 (2015)

Background. Deeper plain networks became *harder to train*, not just to generalise: adding layers raised the training error. He et al. let every block learn a correction to its input, $y = x + F(x)$, through a shortcut [9]; gradients then flow through the shortcuts, and networks of 50, 101, even 1 000 layers train well [10].

As built. Stem: Conv2D 64 7×7 stride 2 + BatchNorm + ReLU, max pool 3×3 stride 2 (224 → 56). Four stages of two basic blocks with 64, 128, 256 and 512 filters; the first block of stages 2–4 has stride 2 and a 1×1 stride-2 convolution with BatchNorm on the shortcut (56 → 28 → 14 →

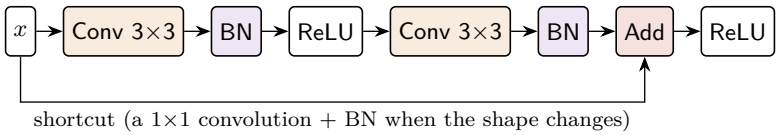


Figure 10: A basic residual block as built on the canvas: $y = \mathrm{ReLU}(x + F(x))$.

7). Global average pooling and the output. 11 181 642 parameters, exactly torchvision's `resnet18` with a 10-class head.

Training it. The most forgiving of the large demos thanks to BatchNorm and the shortcuts. At 224×224 it fits a T4 at batch 64–128. For CIFAR-10 at 32×32 the stem shrinks the image too early (32 → 8 before the first block); the usual CIFAR variant replaces the stem with a 3×3 stride-1 convolution and drops the max pooling, an easy edit on the canvas.

13.7 MobileNet v1 (2017)

Background. For phones, Howard et al. factorised a convolution into a *depthwise* 3×3 convolution (one filter per channel, spatial only) and a *pointwise* 1×1 convolution (mixing channels) [13]. The cost drops by a factor of about $1/N + 1/k^2$ (N output channels, $k = 3$): roughly eight to nine times fewer computations.

As built. Conv2D 32 3×3 stride 2 + BN + ReLU, then 13 depthwise separable blocks (Depthwise Conv2D 3×3 + BN + ReLU, Conv2D 1×1 + BN + ReLU) with 64, 128, 128, 256, 256, 512 (×6), 1 024, 1 024 channels; five of them have stride 2 (224 → 7); global average pooling and the output. 3 217 226 parameters (4.2 million with 1 000 classes, as in the paper). Width and resolution multipliers of the paper can be imitated by changing the filter counts and the input size.

Training it. Light and fast; SGD or Adam 0.001 with BatchNorm works well. Depthwise convolutions are memory-bound on GPUs, so it is not proportionally faster to *train*; its advantage is at inference on small devices.

13.8 DenseNet, small CIFAR version (2017)

Background. Huang et al. connected each layer to *all* earlier layers of its block by concatenation [15]: features are reused instead of re-learnt, so each layer can be narrow (the "growth rate" k, here 12 channels).

As built. Conv2D 24 3×3; three dense blocks of four layers (BatchNorm → ReLU → Conv2D 12 3×3, concatenated to everything before); between blocks a transition (BatchNorm → 1×1 Conv2D halving the channels → average pool 2×2). The channels go 24 → 72 → 36 → 84 → 42 → 90 while the image goes 32 → 16 → 8; BatchNorm, ReLU, global pool and output. Only 76 810 parameters.

Differences from the paper. The paper's CIFAR models are much deeper (for example 100 layers with bottleneck 1×1 convolutions inside each layer, about 0.8 million parameters); the demo is a small teaching version. The paper's transitions have a ReLU after their BatchNorm; the demo's do not.

Training it. CIFAR-10 built in, Adam 0.001, flips and crops, 50+ epochs; small but deep, so batch 64–128 fits easily.

13.9 Transformer encoder on image rows (2017)

Background. The Transformer of Vaswani et al. replaced recurrence with self-attention: every token computes a weighted mix of all tokens, the weights coming from query–key similarities [40].

Its encoder is a stack of attention and two-layer MLP blocks with residual connections and layer normalisation [1].

As built. A teaching version on MNIST: To tokens (*rows*) makes each of the 28 rows a token of 28 values; Dense 64 embeds them; Class token + position gives 29 tokens of 64; two pre-norm encoder blocks with 4 heads, an MLP of 128 (GELU [11]) and dropout 0.1; LayerNorm, the class token, output. 71 498 parameters.

Training it. MNIST, Adam 0.001, 10–20 epochs: it reaches high-90s accuracy and shows that attention can learn image structure from a sequence of rows. Change the number of heads (64 must stay divisible by it) or blocks and compare.

13.10 ViT-Base/16 (2020)

Background. Dosovitskiy et al. showed that a plain Transformer encoder on image patches, "an image is worth 16×16 words", matches the best CNNs when pre-trained on enough data [6].

As built. Conv2D 768 filters 16×16 stride 16 (the patch embedding: 224 → 14×14), To tokens (196 tokens of 768), Class token + position (197 tokens), 12 encoder blocks (12 heads, MLP 3 072), LayerNorm, class token, output. 85 806 346 parameters, exactly torchvision's `vit_b_16` with a 10-class head (86.6 million with 1 000 classes).

Training it. ViTs lack the built-in assumptions of convolutions (locality, translation equivariance) and need much data or strong augmentation and regularisation [37, 39]. From scratch on a small dataset it does worse than ResNet-18. Use a low learning rate (0.0001 or less; AdamW with warm-up is the usual recipe [24, 8], not built in), batch 32–64 on a T4 with mixed precision.

13.11 ViT-Tiny for CIFAR-10

Background. Small ViTs in the spirit of DeiT-Ti [39] (embedding 192, 3 heads) make transformers trainable on small images in minutes.

As built. Conv2D 192 4×4 stride 4 (32 → 8×8 = 64 patches), To tokens, Class token + position (65 tokens), 6 encoder blocks with 3 heads, MLP 384 and dropout 0.1, LayerNorm, class token, output. 1 806 538 parameters. (DeiT-Ti itself has 12 blocks and an MLP of 768.)

Training it. CIFAR-10 with flips and crops (augmentation matters a lot for ViTs), Adam 0.001 or a little lower, 50+ epochs. It usually stays below the Small CNN on CIFAR-10 when trained from scratch for a short time: a useful experiment on the data hunger of transformers.

13.12 ViT-g/14, one billion parameters (2022)

Background. Zhai et al. studied how ViTs scale with model size, data and compute, training up to two billion parameters on billions of images [45]. ViT-g/14 ("giant") is their one-billion-parameter model.

As built. Conv2D 1 408 filters 14×14 stride 14 (224 → 16×16 = 256 patches), Class token + position (257 tokens), 40 encoder blocks with 16 heads and MLP 6 144, LayerNorm, class token, output: 1 011 216 522 parameters. Each encoder block alone has 25 250 176 parameters (attention $4d^2 + 4d = 7\,935\,488$; MLP $2 \cdot 1408 \cdot 6144 + 6144 + 1408 = 17\,309\,056$; two LayerNorms 5 632). The canvas holds about 290 blocks; the layer table groups them into Embedding, 40 encoder blocks, Final norm and Classifier.

Training it. About 16 GB of fixed state for Adam (Section 6): it does *not* fit a 16 GB T4 (Kaggle, g4dn.xlarge). A 48 GB RTX A6000 fits it with batches of roughly 16–32 at 224×224 (estimate), at around 1.5 TFLOP of compute per training image; a checkpoint with the optimiser state is about 12 GB and the final `model.pt` about 4 GB, both copied over the network. From scratch on a small

dataset it overfits badly and scores below ResNet-18; its value here is to see how a transformer scales (compare it block by block with ViT-Base) and to measure memory and speed.

13.13 U-Net (2015)

Background. Ronneberger et al. designed U-Net for biomedical image segmentation with few training images [32]: a contracting path captures context, an expanding path recovers resolution, and skip connections hand the fine detail of each encoder level to the decoder level of the same size.

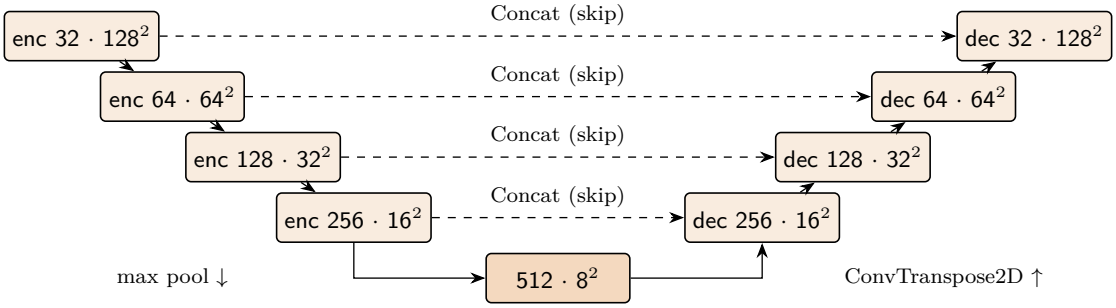


Figure 11: The U shape as built: channels · image size per level, 128 → 8 pixels and back, with a skip at every level.

As built. Input 3×128×128. Encoder: four levels of two 3×3 convolutions (ReLU) with 32, 64, 128, 256 filters, each followed by 2×2 max pooling; bottleneck: two convolutions with 512 filters at 8×8. Decoder: at each level a 2×2 stride-2 ConvTranspose2D halving the channels, Concat with the encoder output of the same size, two 3×3 convolutions; Segmentation output (1×1) with two classes. 7 760 130 parameters.

Differences from the paper. The paper starts at 64 filters (1 024 at the bottom, about 31 million parameters) and uses unpadded convolutions, cropping the encoder maps before concatenating; the demo uses padding "same", so sizes match without cropping.

Training it. Dataset → Segmentation, with an image column and a mask column (Hugging Face) or images and masks folders (Kaggle); the class list maps mask values to classes. Flips and rotations are applied to the image and its mask together. Watch the mIoU curve and the predicted masks in Results.

13.14 Attention U-Net (2018)

Background. Oktay et al. added *attention gates* to U-Net's skip connections: the decoder's coarser signal decides which regions of the encoder features matter, suppressing irrelevant background [28].

As built. U-Net as above, but before each Concat the skip features x pass a gate: g (the up-sampled decoder signal) and x are each projected by a 1×1 convolution to half the channels, added, passed through ReLU and a 1×1 convolution to one channel with a sigmoid; the resulting map $\psi \in [0,1]^{H \times W}$ multiplies x (Multiply, one-channel map times image). 7 847 894 parameters.

Difference from the paper. The paper takes the gating signal from the coarser decoder level and resamples it; the demo uses the up-sampled decoder features, which already have the skip's size.

13.15 MSG-Former: an NN Builder original

MSG-Former (*Multi-Scale Gated transFormer*) is the one demo that does not come from a paper: it was designed for NN Builder as a small, fast network for 32×32 colour images that combines three published ideas in one design. It is an untested proposal, not a benchmark result: every number below (shapes, parameters, receptive fields, compute) is exact, but how well it classifies is for its first training runs to show.

13.15.1 The idea

Convolutional networks and transformers are good at different things. A convolution looks at a small neighbourhood and is very efficient at finding local texture and edges, but needs many layers before one output can relate two distant parts of the image. Self-attention relates every position to every other in one step, but on raw pixels it is expensive and, without the built-in assumptions of convolutions, needs much more data [6, 39]. MSG-Former therefore works in two phases:

1. **Local, multi-scale, gated** (32×32 and 16×16 pixels): convolution stages that look at three scales at once, weigh their channels by how useful they are, and keep a residual path;
2. **Global** (an 8×8 grid): the feature map becomes 64 tokens, and two Transformer encoder blocks let every region attend to every other before the class token makes the prediction.

The same convolution-first, attention-later order is used by hybrid models such as the ResNet–ViT hybrid of the ViT paper [6], CoAtNet [5] and MobileViT [26]; MSG-Former's own contribution is the form of its convolution stage.

13.15.2 The multi-scale gated residual block

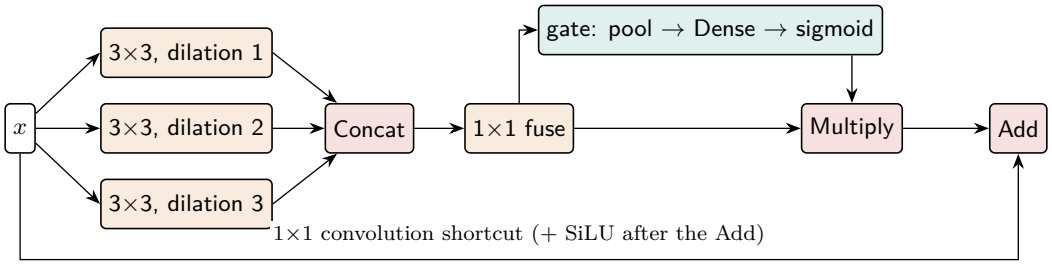


Figure 12: The multi-scale gated residual block. Each 3×3 branch is Conv2D (no bias) → BatchNorm → SiLU; the fuse is a 1×1 Conv2D → BatchNorm → ReLU; the gate is Global pool → Dense ($C/4$, ReLU) → Dense (C, sigmoid).

For an input x with C_{in} channels, a branch width b and an output width C, the block computes

$$u_r = \mathrm{SiLU}\big(\mathrm{BN}(W_r *_r x)\big), \qquad r \in \{1,2,3\} \quad (*_r :\ 3{\times}3 \text{ convolution with dilation } r), \tag{4}$$

$$v = \mathrm{ReLU}\big(\mathrm{BN}(W_f * [u_1; u_2; u_3])\big) \qquad (1{\times}1 \text{ fuse of the } 3b \text{ concatenated channels to } C), \tag{5}$$

$$g = \sigma\big(W_2\ \mathrm{ReLU}(W_1\ \mathrm{GAP}(v) + c_1) + c_2\big) \in (0,1)^C, \tag{6}$$

$$y = \mathrm{SiLU}\big(g \odot v + W_s * x\big) \qquad (W_s :\ 1{\times}1 \text{ shortcut, } C_{\text{in}} \to C). \tag{7}$$

Each part has a reason:

Three dilations, same cost. A 3×3 kernel with dilation r covers $(2r+1) \times (2r+1)$ pixels with the same nine weights: 3×3, 5×5 and 7×7 here [44]. The block sees fine detail and wider context at the same resolution and the price of three ordinary 3×3 convolutions, like an Inception module [38] without the cost of large kernels, or the atrous spatial pyramid of

DeepLab [2] inside a classifier. Mixing the rates 1, 2 and 3 (rather than only large ones) keeps every pixel covered and avoids the "gridding" gaps that stacked dilated convolutions can leave [41].

1×1 fuse. Concatenation gives $3b$ channels; the 1×1 convolution mixes the scales and sets the block's width.

Squeeze-and-excitation gate. Global average pooling summarises each channel over the whole image; two small Dense layers (reduction 4) turn that summary into one weight in $(0, 1)$ per channel, and Multiply scales the channels with it, so the block can emphasise the scales and features that matter for this image [14]. The original SE networks use reduction 16; with 64 and 128 channels, 4 keeps the bottleneck from becoming too narrow (16 and 32 units).

Residual shortcut. As in ResNet [9], the block learns a correction to its input; because the channel count changes ($32 \to 64$, $64 \to 128$), the shortcut is a 1×1 projection.

SiLU. A smooth activation [31] in the branches and after the sum; the fuse keeps ReLU.

13.15.3 The whole network

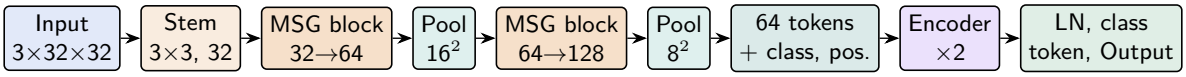


Figure 13: MSG-Former from left to right: local multi-scale stages at 32×32 and 16×16, then global attention on an 8×8 grid.

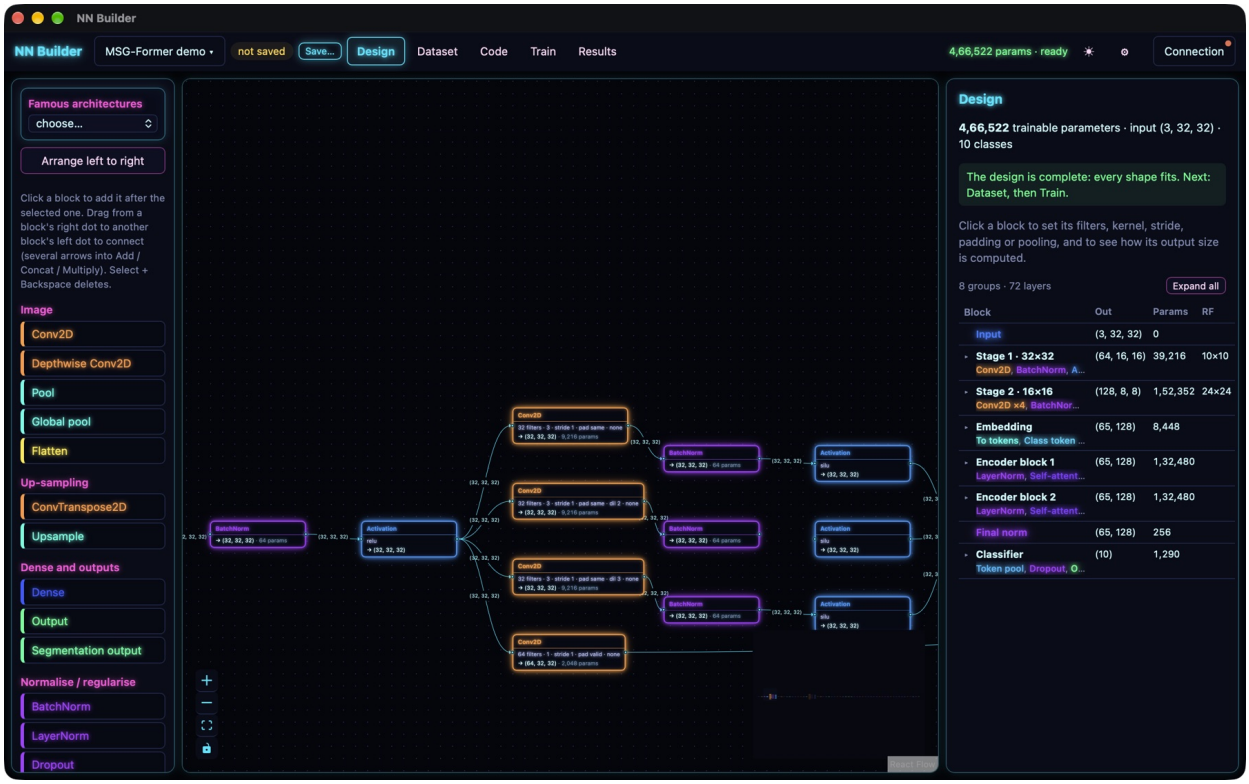


Figure 14: MSG-Former on the canvas, zoomed in on the first block: the stem's activation feeds the three dilated 3×3 convolutions (dilation 1, 2, 3, each with BatchNorm and SiLU) and, at the bottom, the 1×1 shortcut; the branches meet in Concat (96 channels). Every arrow shows its tensor shape.

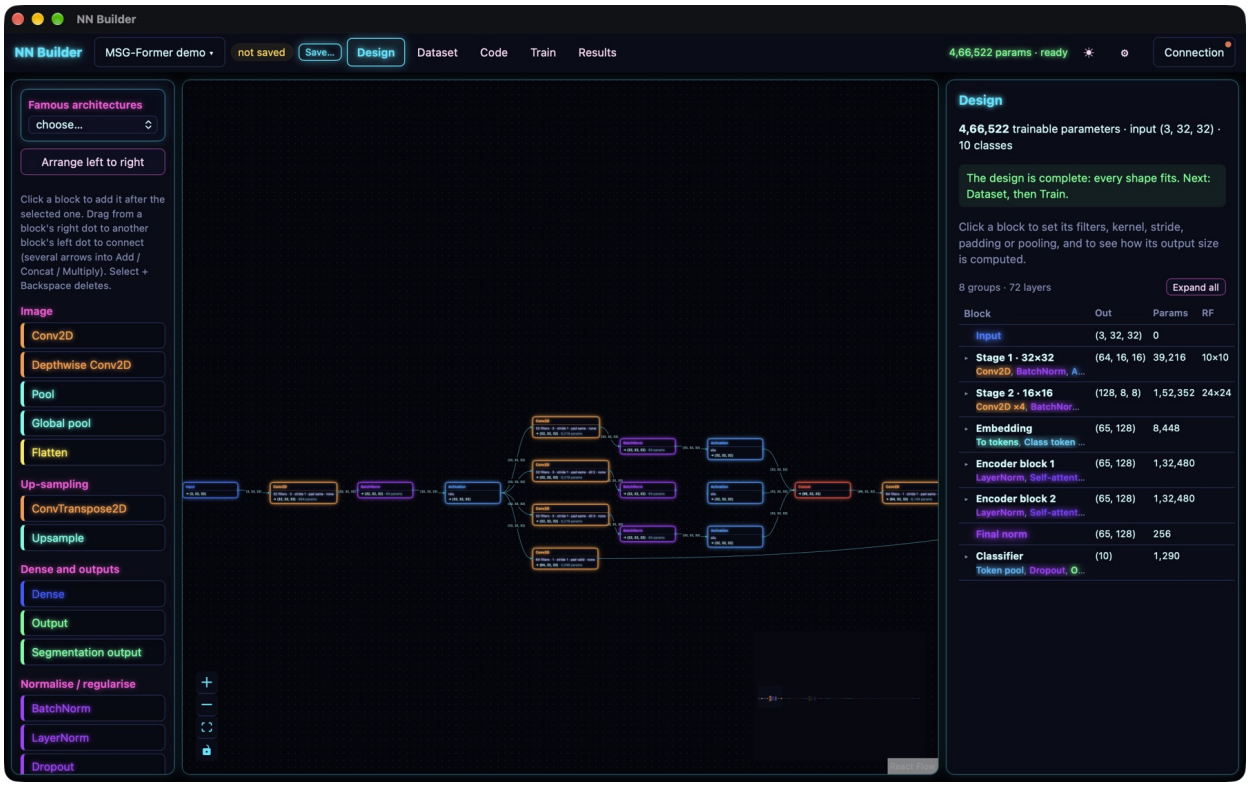


Figure 15: The whole design opened from the demo: 72 blocks, 466 522 parameters, "ready". The layer table groups them as Input, Stage 1 · 32×32, Stage 2 · 16×16, Embedding, Encoder block 1 and 2, Final norm and Classifier, each with its output, parameters and receptive field.

Table 7 lists every layer as the application computes it (with 10 classes). The receptive field (RF) is given in input pixels.

Table 7: MSG-Former layer by layer.

Stage	Layer	Output	Parameters	RF
Input	image	3×32×32	0	1
Stem	Conv2D 32, 3×3, no bias; BatchNorm; ReLU	32×32×32	864 + 64	3
Block 1	branch $r = 1$: Conv2D 32 3×3; BatchNorm; SiLU	32×32×32	9 216 + 64	5
	branch $r = 2$: Conv2D 32 3×3 dilation 2; BatchNorm; SiLU	32×32×32	9 216 + 64	7
	branch $r = 3$: Conv2D 32 3×3 dilation 3; BatchNorm; SiLU	32×32×32	9 216 + 64	9
	Concat	96×32×32	0	9
	fuse: Conv2D 64 1×1; BatchNorm; ReLU	64×32×32	6 144 + 128	9
	gate: Global pool; Dense 16 ReLU; Dense 64 sigmoid	64	1 040 + 1 088	
	Multiply (gate × fused map)	64×32×32	0	
	shortcut: Conv2D 64 1×1, no bias	64×32×32	2 048	
	Add; SiLU	64×32×32	0	
	Pool max 2×2	64×16×16	0	10
Block 2	branches $r = 1, 2, 3$: Conv2D 64 3×3; BatchNorm; SiLU	3 × 64×16×16	3 × (36 864 + 128)	14, 18, 22
	Concat	192×16×16	0	22

Stage	Layer	Output	Parameters	RF
	fuse: Conv2D 128 1×1; BatchNorm; ReLU	128×16×16	24 576 + 256	22
	gate: Global pool; Dense 32 ReLU; Dense 128 sigmoid	128	4 128 + 4 224	
	Multiply; shortcut Conv2D 128 1×1; Add; SiLU	128×16×16	8 192	
	Pool max 2×2	128×8×8	0	24
Embedding	To tokens (patches): every cell of the 8×8 map	64×128	0	
	Class token + position embeddings	65×128	8 448	
Encoder 1	LayerNorm; Self-attention (4 heads, dropout 0.1); Dropout; Add	65×128	256 + 66 048	global
	LayerNorm; Dense 256 GELU; Dropout; Dense 128; Dropout; Add	65×128	256 + 33 024 + 32 896	
Encoder 2	the same	65×128	132 480	
Head	LayerNorm; Token pool (class token); Dropout 0.2; Output	10	256 + 1 290	
	Total		**466 522**	

Where the parameters are.

Part	Parameters	Share
Stem	928	0.2%
Block 1 (branches 27 840, fuse 6 272, gate 2 128, shortcut 2 048)	38 288	8.2%
Block 2 (branches 110 976, fuse 24 832, gate 8 352, shortcut 8 192)	152 352	32.7%
Class token and positions	8 448	1.8%
Two encoder blocks (attention 66 048, MLP 65 920, norms 512 each)	264 960	56.8%
Final LayerNorm and output	1 546	0.3%
Total	466 522	100%

These counts were derived by hand, agree with the application's engine, are pinned in its automatic tests, and are confirmed against a real PyTorch build of the network on every change (the comparison on GitHub, Section 6).

Where the computation is. Counting multiply–accumulate operations (MACs) of the convolutions, the Dense layers and attention for one 32×32 image (BatchNorm, activations and pooling are negligible):

Part	MACs per image
Stem ($9 \cdot 3 \cdot 32 \cdot 32^2$)	0.88 M
Block 1: branches $3 \cdot (9 \cdot 32 \cdot 32 \cdot 32^2)$ 28.3 M, fuse 6.3 M, shortcut 2.1 M	36.7 M
Block 2: branches $3 \cdot (9 \cdot 64 \cdot 64 \cdot 16^2)$ 28.3 M, fuse 6.3 M, shortcut 2.1 M	36.7 M
Encoders: per block projections $4Nd^2$ 4.3 M, attention $2N^2d$ 1.1 M, MLP $2Nd \cdot 256$ 4.3 M ($N = 65$, $d = 128$)	19.2 M
Total	about 93.5 M

About 94 million multiply–accumulates (roughly 0.19 GFLOP) per image: the convolution stages do about 80% of the work with about 41% of the parameters, the encoders the reverse (21% of the work, 57% of the parameters). Because pooling halves the image before block 2 while the width

doubles, both blocks cost almost exactly the same. Attention is cheap here because there are only 65 tokens: its N^2 term is 1.1 M MACs per block. Training costs about three times the forward pass, so one epoch of CIFAR-10 (50 000 images) is roughly 1.4×10^{13} MACs, a few minutes on a T4.

Receptive field. Before the transformer each token sees a 24×24 pixel region of the 32×32 image (the RF column of Table 7), and neighbouring tokens are 4 pixels apart, so their regions overlap heavily. The first encoder block then makes the receptive field global: every token, and the class token, can draw on every part of the image.

13.15.4 Training it

1. Open MSG-Former from the Projects page and save it (Cmd+S). It opens with built-in CIFAR-10 at its own size.
2. Train tab: Adam, learning rate 0.001 (0.0005 if the loss is unstable), batch 128, 30–60 epochs; augmentation: horizontal flip and random crop with 4 pixels of padding (transformer layers in particular profit from augmentation [37]).
3. Train on Kaggle or any GPU: by a rough estimate it needs about 1 GB of GPU memory at batch 128 (mostly the activations of the two convolution stages kept for the backward pass), so even small GPUs are enough.
4. Compare with the Small CNN (551 018 parameters, convolution only) and ViT-Tiny (1 806 538 parameters, attention only), trained with the same settings, in the Runs table.

13.15.5 Experiments to run (ablations)

Each is a small edit on the canvas, and together they show which part of the design earns its place:

	Change on the canvas	**Parameters**
A	Remove both gates (delete Global pool, the two Dense layers and Multiply; connect the fuse to Add)	456 042
B	All dilations 1 (three identical-scale branches)	466 522
C	No transformer: Global pool after the second pooling, then Output	192 858
D	A third encoder block	598 970
E	Token pool *mean* instead of the class token	466 522
F	Wider: branch filters 64 and 128	larger

(Counts for A, C and D follow from the tables above: A removes 2 128 + 8 352; C removes the embedding, encoders, final norm and the 128-input head and adds a 128-input head; D adds one encoder block of 132 480. The application shows the exact count after each edit.) If B matches the full model, the multi-scale branches are not helping on this data; if C matches it, the attention is not; and so on.

13.15.6 Adapting it

- **Larger images** (for example 64×64 or 224×224): add a stride-2 stem (or a third block with pooling) so that the grid reaching the transformer stays around 8×8 to 14×14; otherwise the number of tokens, and the N^2 attention cost, grows with the image.
- **More classes:** only the output layer changes ($128 \cdot K + K$ parameters).

- **Segmentation:** the two convolution stages can serve as an encoder in a U-Net-style design, with ConvTranspose2D decoders and skips from the blocks.

13.15.7 Status

The design, its shapes and its parameter count are verified (by hand, by the application's engine and against PyTorch). Its accuracy is not: no training results exist yet. Treat it as a starting point for experiments, and record the results of the comparisons above.

14 Troubleshooting

"design has problems". Open Design; the red block says why (Section 5.1). Blocks after it are greyed out until it is fixed.

"dataset incomplete" or no classes. The class list is empty (Hugging Face: **Read info** and a ClassLabel column; Kaggle: type the class folder names). Text datasets cannot be used.

The one-batch check stops the run. A layer's real shape differed from the application's (for example images of an unexpected size in a custom dataset). The message names the layer; check the image size and channels in the Dataset tab.

"CUDA out of memory". Halve the batch size; turn mixed precision on; use a smaller input size or network (Section 6).

Kaggle run fails to start. The account must be phone-verified for GPU and internet access; the token must be current.

SSH machine does not answer. Password machines: press **Connect** again (the login lasts a few hours); key machines: **Test**. A running training is not affected.

EC2: "no public address" / ssh never answers. Give the instance a public IP (or an Elastic IP), allow port 22 in its security group, and use its key file. A new account may have a vCPU quota of 0 for G instances: request an increase.

EC2 spot: the run says "interrupted". AWS took the capacity back; with a persistent request and behaviour *stop* it resumes by itself. To continue now, use **Continue on another machine** (needs the checkpoint copy on the Mac).

A demo disappeared. Unsaved demos are discarded when another demo or project is opened or the application quits; save them with Cmd+S.

Training is unstable (loss jumps or becomes NaN). Lower the learning rate (by 10×), keep mixed precision on (the script scales the loss), add BatchNorm / LayerNorm, check that the labels match the class list.

High training accuracy, low validation accuracy. Overfitting: augmentation, dropout, fewer parameters (Global pool instead of Flatten + large Dense), more data, fewer epochs.

15 Privacy and security

- Tokens and keys live only in the system keychain; they are passed to one call at a time, and written on a remote machine only when a run needs them, as files readable by their owner.
- The design computer never downloads datasets or runs training; it reads dataset descriptions only.

- Unsaved demos are kept in the application's own folder and deleted when no longer needed; projects live where you choose.
- Updates come only from the project's GitHub releases and are checked against the published SHA-256 checksum.

A Glossary

Activation function a non-linear function applied element-wise (ReLU, GELU, ...).

Batch the images processed together in one training step.

Channel one "layer" of an image or feature map (3 for RGB; a convolution with 64 filters produces 64).

Checkpoint a saved training state (weights, optimiser, position) from which a run can resume.

Epoch one pass over the whole training set.

Feature map the output of a convolution: one image per filter.

Kernel / filter the small window of weights a convolution slides over the image.

mIoU mean over the classes of (overlap / union) of predicted and true masks.

Padding pixels added around the image so the window can reach the borders.

Parameter a trainable number (weight or bias).

Receptive field the input region one output value depends on.

Spot instance spare cloud capacity at a lower price that the provider can take back.

Stride how far the window moves at each step.

Token one element of a sequence a transformer works on (here: an image patch or row).

References

[1] J. L. Ba, J. R. Kiros and G. E. Hinton. Layer normalization. 2016. arXiv:1607.06450.

[2] L.-C. Chen, G. Papandreou, F. Schroff and H. Adam. Rethinking atrous convolution for semantic image segmentation. 2017. arXiv:1706.05587.

[3] T. Chen, B. Xu, C. Zhang and C. Guestrin. Training deep nets with sublinear memory cost. 2016. arXiv:1604.06174.

[4] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan and Q. V. Le. AutoAugment: learning augmentation policies from data. 2018. arXiv:1805.09501.

[5] Z. Dai, H. Liu, Q. V. Le and M. Tan. CoAtNet: marrying convolution and attention for all data sizes. 2021. arXiv:2106.04803.

[6] A. Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. 2020. arXiv:2010.11929.

[7] V. Dumoulin and F. Visin. A guide to convolution arithmetic for deep learning. 2016. arXiv:1603.07285.

[8] P. Goyal et al. Accurate, large minibatch SGD: training ImageNet in 1 hour. 2017. arXiv:1706.02677.

[9] K. He, X. Zhang, S. Ren and J. Sun. Deep residual learning for image recognition. 2015. arXiv:1512.03385.

[10] K. He, X. Zhang, S. Ren and J. Sun. Identity mappings in deep residual networks. 2016. arXiv:1603.05027.

[11] D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs). 2016. arXiv:1606.08415.

[12] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. 2012. arXiv:1207.0580.

[13] A. G. Howard et al. MobileNets: efficient convolutional neural networks for mobile vision applications. 2017. arXiv:1704.04861.

[14] J. Hu, L. Shen, S. Albanie, G. Sun and E. Wu. Squeeze-and-excitation networks. 2017. arXiv:1709.01507.

[15] G. Huang, Z. Liu, L. van der Maaten and K. Q. Weinberger. Densely connected convolutional networks. 2016. arXiv:1608.06993.

[16] S. Ioffe and C. Szegedy. Batch normalization: accelerating deep network training by reducing internal covariate shift. 2015. arXiv:1502.03167.

[17] D. P. Kingma and J. Ba. Adam: a method for stochastic optimization. 2014. arXiv:1412.6980.

[18] A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009 (not on arXiv). `https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf`.

[19] A. Krizhevsky, I. Sutskever and G. E. Hinton. ImageNet classification with deep convolutional neural networks. NeurIPS 2012 (not on arXiv). `https://papers.nips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html`.

[20] A. Krizhevsky. One weird trick for parallelizing convolutional neural networks. 2014. arXiv:1404.5997.

[21] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner. Gradient-based learning applied to document recognition. Proceedings of the IEEE 86(11), 1998 (not on arXiv). doi:10.1109/5.726791.

[22] Q. Lhoest et al. Datasets: a community library for natural language processing. 2021. arXiv:2109.02846.

[23] M. Lin, Q. Chen and S. Yan. Network in network. 2013. arXiv:1312.4400.

[24] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. 2017. arXiv:1711.05101.

[25] W. Luo, Y. Li, R. Urtasun and R. Zemel. Understanding the effective receptive field in deep convolutional neural networks. 2017. arXiv:1701.04128.

[26] S. Mehta and M. Rastegari. MobileViT: light-weight, general-purpose, and mobile-friendly Vision Transformer. 2021. arXiv:2110.02178.

[27] P. Micikevicius et al. Mixed precision training. 2017. arXiv:1710.03740.

[28] O. Oktay et al. Attention U-Net: learning where to look for the pancreas. 2018. arXiv:1804.03999.

[29] A. Paszke et al. PyTorch: an imperative style, high-performance deep learning library. 2019. arXiv:1912.01703.

[30] S. Rajbhandari, J. Rasley, O. Ruwase and Y. He. ZeRO: memory optimizations toward training trillion parameter models. 2019. arXiv:1910.02054.

[31] P. Ramachandran, B. Zoph and Q. V. Le. Searching for activation functions. 2017. arXiv:1710.05941.

[32] O. Ronneberger, P. Fischer and T. Brox. U-Net: convolutional networks for biomedical image segmentation. 2015. arXiv:1505.04597.

[33] O. Russakovsky et al. ImageNet large scale visual recognition challenge. 2014. arXiv:1409.0575.

[34] R. R. Selvaraju et al. Grad-CAM: visual explanations from deep networks via gradient-based localization. 2016. arXiv:1610.02391.

[35] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. 2014. arXiv:1409.1556.

[36] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. JMLR 15, 2014 (not on arXiv; see also [12]). `https://jmlr.org/papers/v15/srivastava14a.html`.

[37] A. Steiner et al. How to train your ViT? Data, augmentation, and regularization in Vision Transformers. 2021. arXiv:2106.10270.

[38] C. Szegedy et al. Going deeper with convolutions. 2014. arXiv:1409.4842.

[39] H. Touvron et al. Training data-efficient image transformers & distillation through attention. 2020. arXiv:2012.12877.

[40] A. Vaswani et al. Attention is all you need. 2017. arXiv:1706.03762.

[41] P. Wang et al. Understanding convolution for semantic segmentation. 2017. arXiv:1702.08502.

[42] H. Xiao, K. Rasul and R. Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. 2017. arXiv:1708.07747.

[43] R. Xiong et al. On layer normalization in the Transformer architecture. 2020. arXiv:2002.04745.

[44] F. Yu and V. Koltun. Multi-scale context aggregation by dilated convolutions. 2015. arXiv:1511.07122.

[45] X. Zhai, A. Kolesnikov, N. Houlsby and L. Beyer. Scaling Vision Transformers. 2021. arXiv:2106.04560.